ITCS213 Computer Organization and Architecture

Class: 1-2026

Sec 2 Fri 9:00-12:00 IT312 (Aj. Joke)
Sec 3 Fri 13:00-16:00 IT312 (Aj. Joke)
Sec 1 Fri 13:00-16:00 IT311 (Aj. Peter)

Unit 3: **ALU** & **Instruction Sets** & **Pipelining**

Lecturer: Dr. Peter M.D. Scully
Faculty of ICT, Mahidol University
peter.scu@mahidol.edu

Slides adapted from Subject Coordinator: Dr. Kanoksak Wattanachote (Ph.D) (金功勳 博士)
kanoksak.wat@mahidol.ac.th

Mahidol University

Announcement: ITCS213

Unit 3: ALU & Pipelining

Announcements

Assembly Programming Tools links available:

ARM: https://cpulator.01xz.net/?sys=arm

MIPS: https://tio.run/ -> (More with "MARS.jar" IDE, from Unit:4)

x86: https://tutorialspoint.com/compile_assembly_online.php

Today we aim to:

1. *See the differences between assembler languages (ARM, MIPS, X86)*
   a. *Understand Division & Branching & Flags*
   b. *Understand Multiplication.*
2. *Recognize ARM instructions from Hex / Binary.*
3. *Get to know what can go wrong with Pipelined Instructions (Hazards)*

Agenda

Part F: ISAs with Assembly (ARM)

Part F.1: Instruction (*Hex/ Bit String*) Interpretation (for ARM)

Part F.2: ISAs with Assembly (MIPS)

Part F.3: ISAs with Assembly (x86)

Part: Types of Pipelining

Part: Pipelined Processors

Part: Pipeline Hazards: *Data Hazards*

Computer Organization Study Process:

How we build understanding — from instruction sets to pipeline hazards

Step 1: ISA Fundamentals

RISC vs CISC, ARM & MIPS & x86 assembly, differences in instructions and registers.

Step 2: Pipelining Concepts

Pipeline stages, arithmetic & instruction pipelining

Step 3: Pipeline Hazards

Data/control hazards, stalls, forwarding, flushing

Step 4: Exercises

ALU assembly programming across ISAs

All steps are covered in this unit (Unit 3).

PART F

Instruction Set Architectures (ISAs):

Writing assembly across instruction sets to see ISAs in action.

Objectives

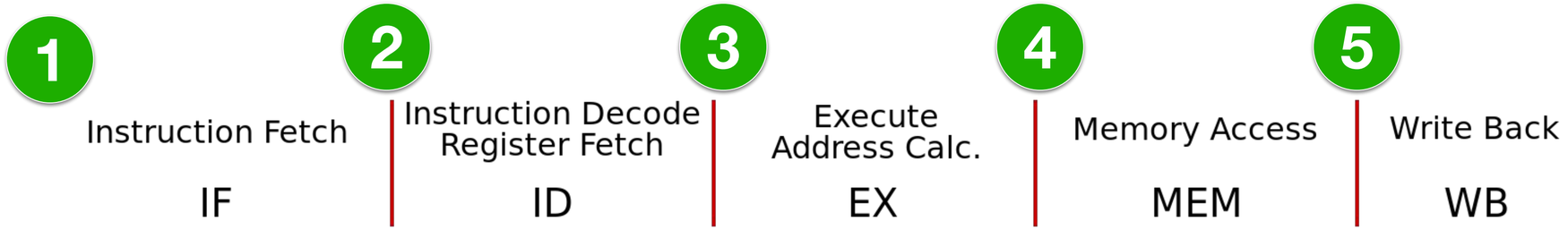
- Understand How Software Interacts with Hardware
- Compare Different ISA Designs (RISC vs. CISC, etc.)
- Proficiency in Low-Level Programming & Debugging
- Foundation for Computer Architecture & Embedded Systems

ISAs

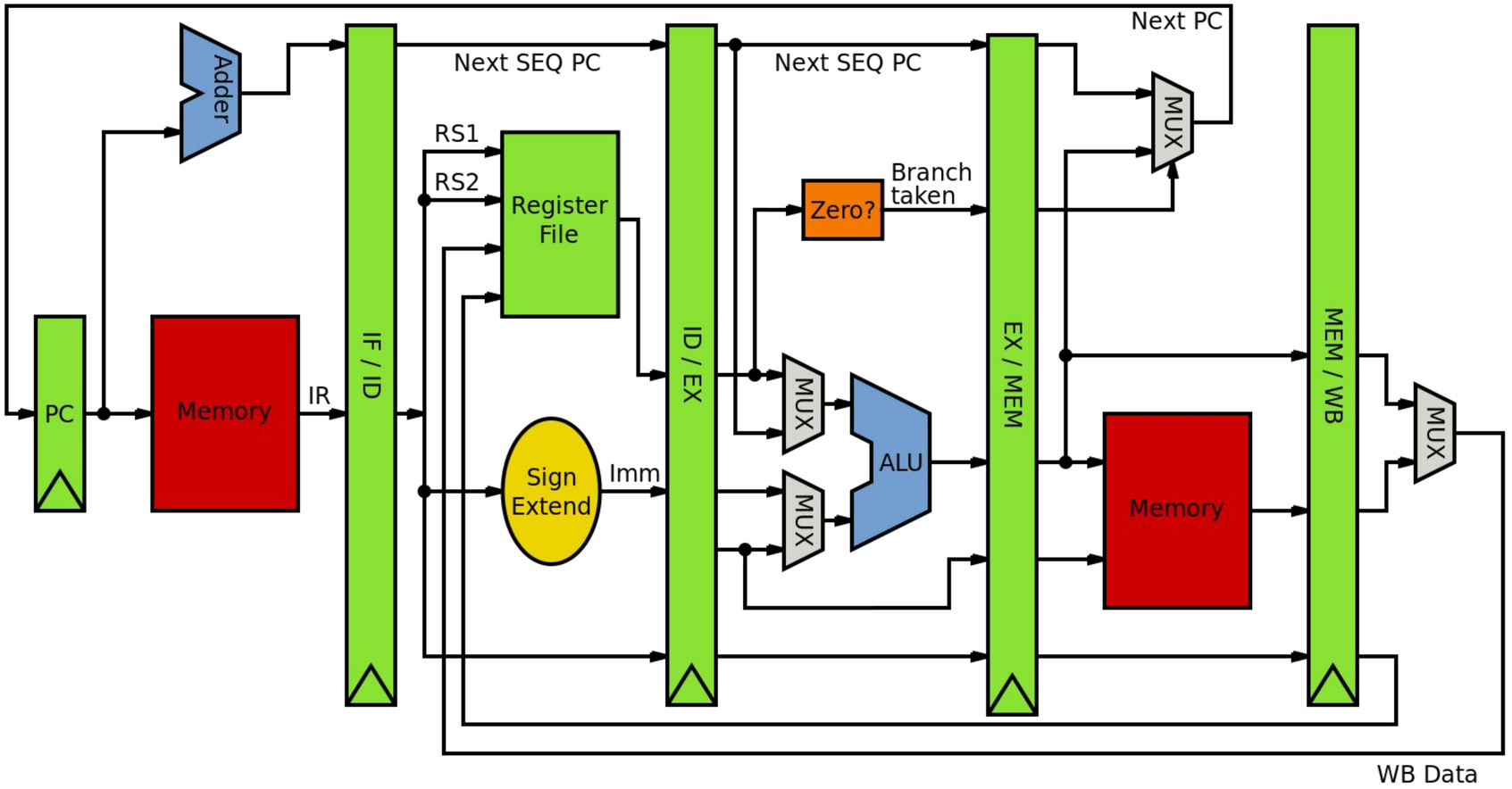
- Design Philosophy: RISC vs CISC

	ARM	MIPS	x86
Type	RISC (Reduced Instruction Set Computer)	RISC	CISC (Complex Instruction Set Computer)
Goal	Fewer, simpler instructions that execute quickly	Same as ARM — simple, load/store model	Many complex instructions, some doing multiple steps in one opcode
Instruction Length	Fixed (usually 32-bit, some 16-bit Thumb)	Fixed 32-bit	Variable (1–15 bytes per instruction)
Registers	16 general-purpose (32-bit in ARMv7, 64-bit in ARMv8)	32 general-purpose	Few general-purpose (x86-64 has 16, but original x86 had only 8)

Pipeline Stages of Instruction Set Architectures: MIPS



5-stage **Pipeline**

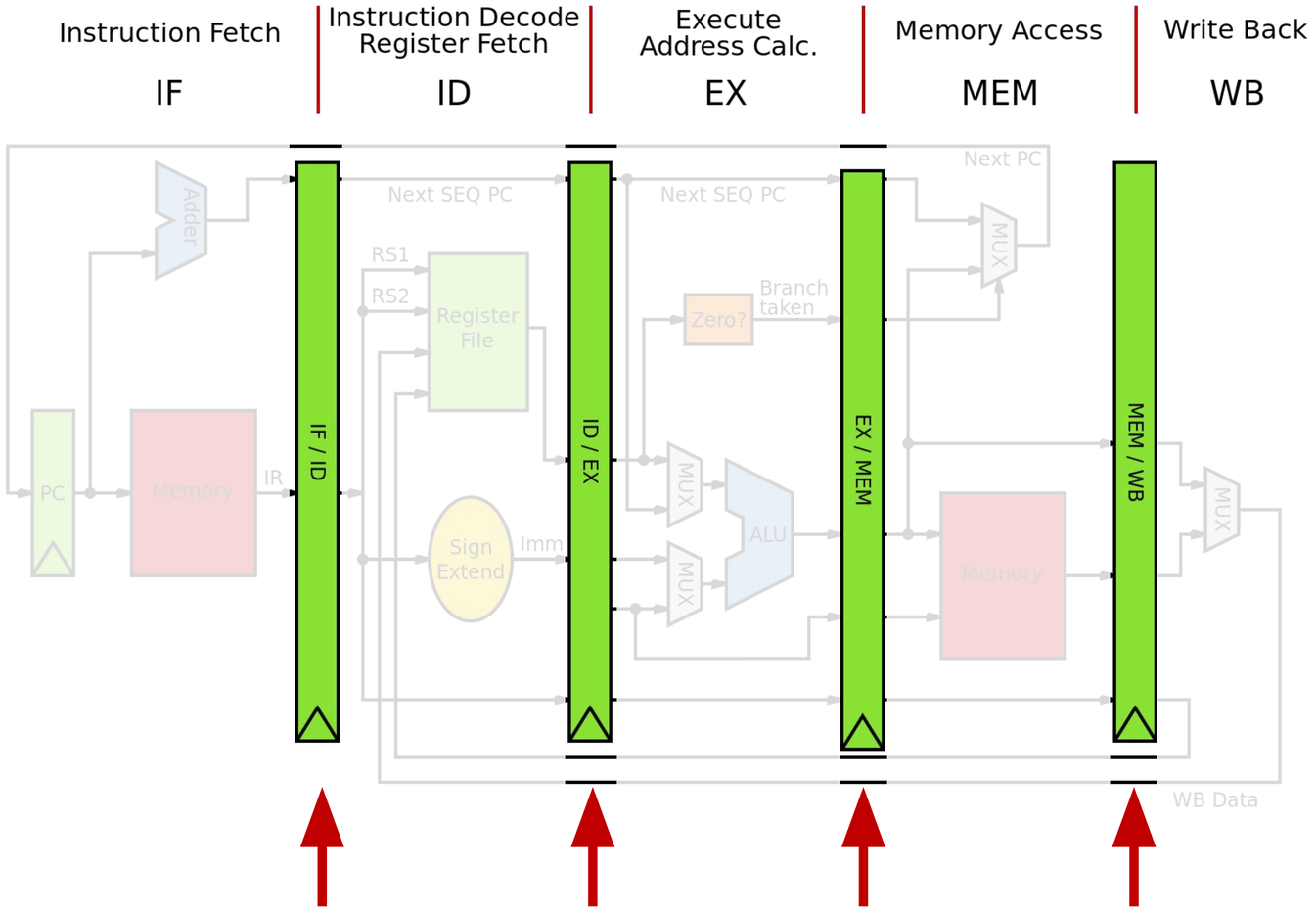


Stages:

1. Instruction Fetch
2. Instruction Decode & Register Fetch
3. Execute
4. Memory Access
5. Write Back

ISA: MIPS

5-stage
Pipeline



Pipeline Registers

(vertical green bars)

They store the **intermediate data** and **control signals** between stages.

- **IF/ID** – Between IF and ID
- **ID/EX** – Between ID and EX
- **EX/MEM** – Between EX and MEM
- **MEM/WB** – Between MEM and WB

Allow **multiple instructions** to be in **different stages at the same time.**

PART F.1

Programming Assembly: for ARM Instruction Set Arch.

(i) Example Code

Writing assembly across instruction sets to see ISAs in action.

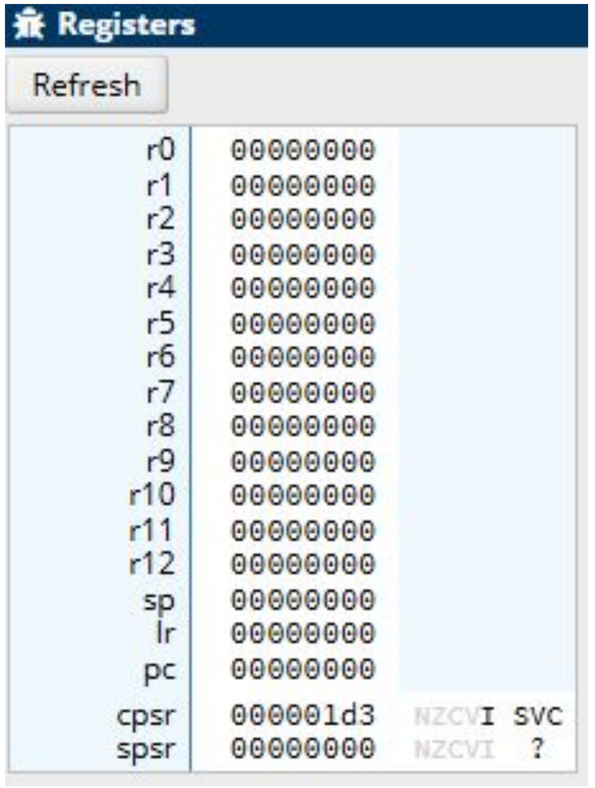
Acorn RISC Machine (ARM)

- ARM originally Acorn RISC Machine, now Advanced RISC Machine.
- ARM is a family of Reduced **Instruction Set** Computing (RISC) architectures used in **CPU**s.
- It is designed for efficiency, making it dominant in:
  - Mobile devices (Smartphones, Tablets: Apple Silicon, Snapdragon).
  - Embedded systems (Raspberry Pi, IoT devices).
  - Modern laptops/desktops (Apple M1/M2, Windows on ARM).

- https://cpulator.01xz.net/?sys=arm

Acorn RISC Machine (ARM)

- Origin: Developed by Acorn Computers in the late 1980s (now ARM Ltd.).
- Instruction format:
  - Also fixed-width, also
  - 3-operand style,
  - but with different register names (R0, R1, etc.).
- Registers: R0–R15 (with special ones like PC, LR, SP).
- Use in industry: Dominates smartphones, tablets, and many embedded systems.



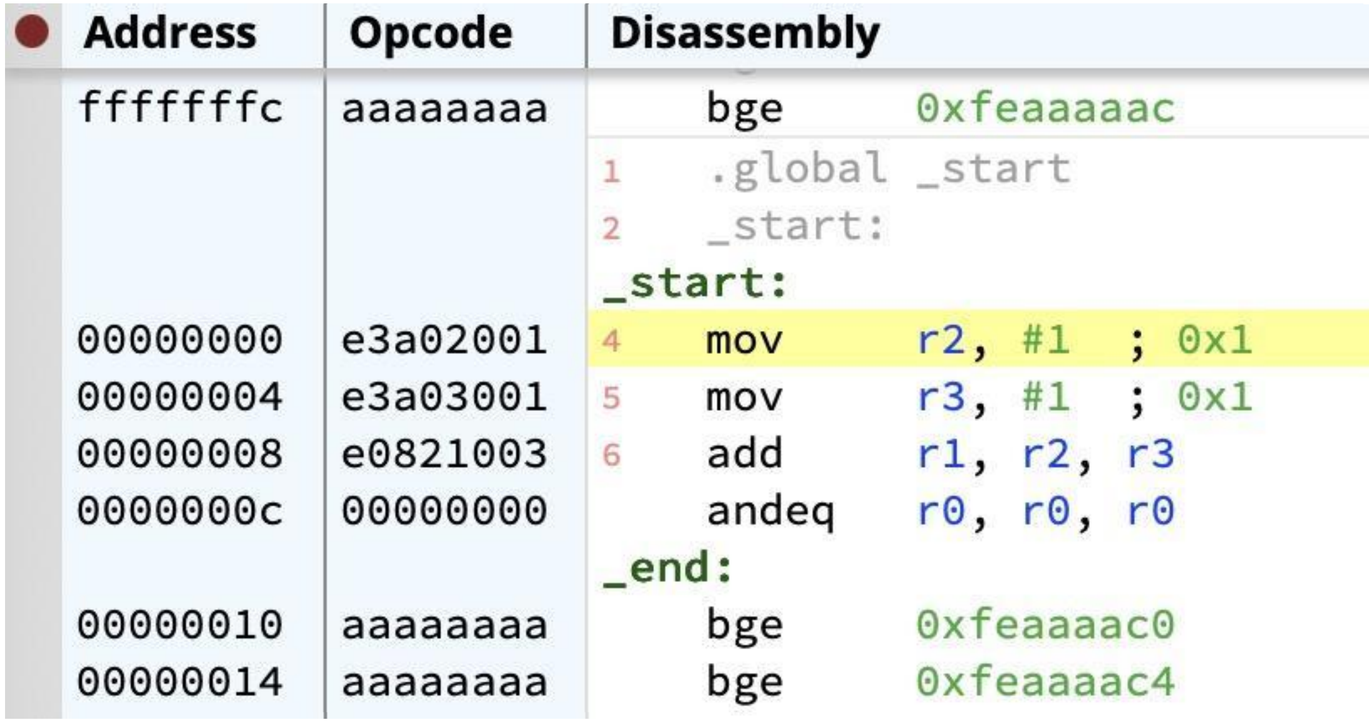
- https://cpulator.01xz.net/?sys=arm

Acorn RISC Machine (ARM)

- For example, write this code in the Editor
  - https://cpulator.01xz.net/?sys=arm

```
.global _start
_start:
    MOV R2, #1
    MOV R3, #1
    ADD R1, R2, R3
```

- Then press “Compile and Load (F5)”
- Then press “Continue”



Acorn RISC Machine (ARM)

- For example, write this code in the Editor
  - https://cpulator.01xz.net/?sys=arm

```
.global _start
_start:
    MOV R2, #5
    MOV R3, #7
    ADD R1, R2, R3
```

- Then press “Compile and Load (F5)”
- Then press “Continue”



Acorn RISC Machine (ARM)

- In **CPUlator** (ARM mode), we won't see "**output**" printed to the console.

- Instead, we watch the register values change as the program runs.
  (Press "Continue")

PART F.1

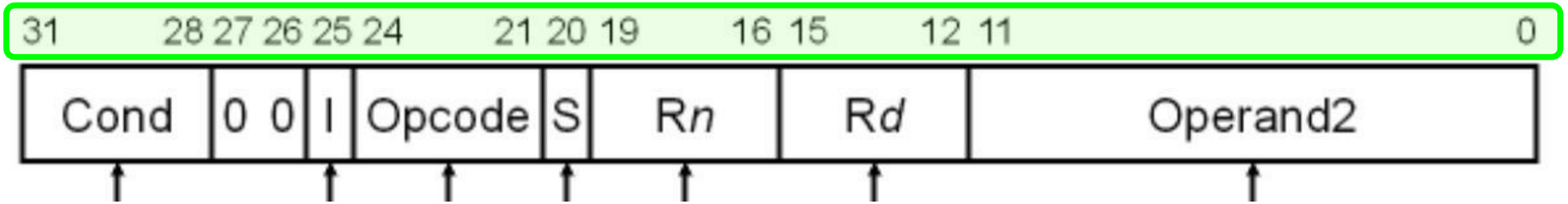
Programming Assembly: for ARM Instruction Set Arch.

(ii) Mapping Bits to Instruction Parts

Writing assembly across instruction sets to see ISAs in action.

ARM: All bits

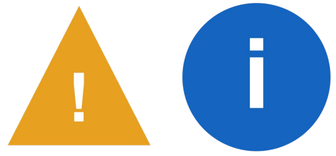
See MyCourses → Unit-3 → Cheat Sheet table (32-bit ARM Instructions)



- *This is the ARM 32-bit Instruction Format.*
- *We'll find out what each of these parts and bits correspond to next.*

Address	Opcode	Disassembly
fffffffc	aaaaaaaa	bge 0xfeaaaaac
		1 .global _start
		2 _start:
		_start:
00000000	e3a02001	4 mov r2, #1 ; 0x1
00000004	e3a03001	5 mov r3, #1 ; 0x1
00000008	e0821003	6 add r1, r2, r3
0000000c	00000000	andeq r0, r0, r0
		_end:
00000010	aaaaaaaa	bge 0xfeaaaac0
00000014	aaaaaaaa	bge 0xfeaaaac4

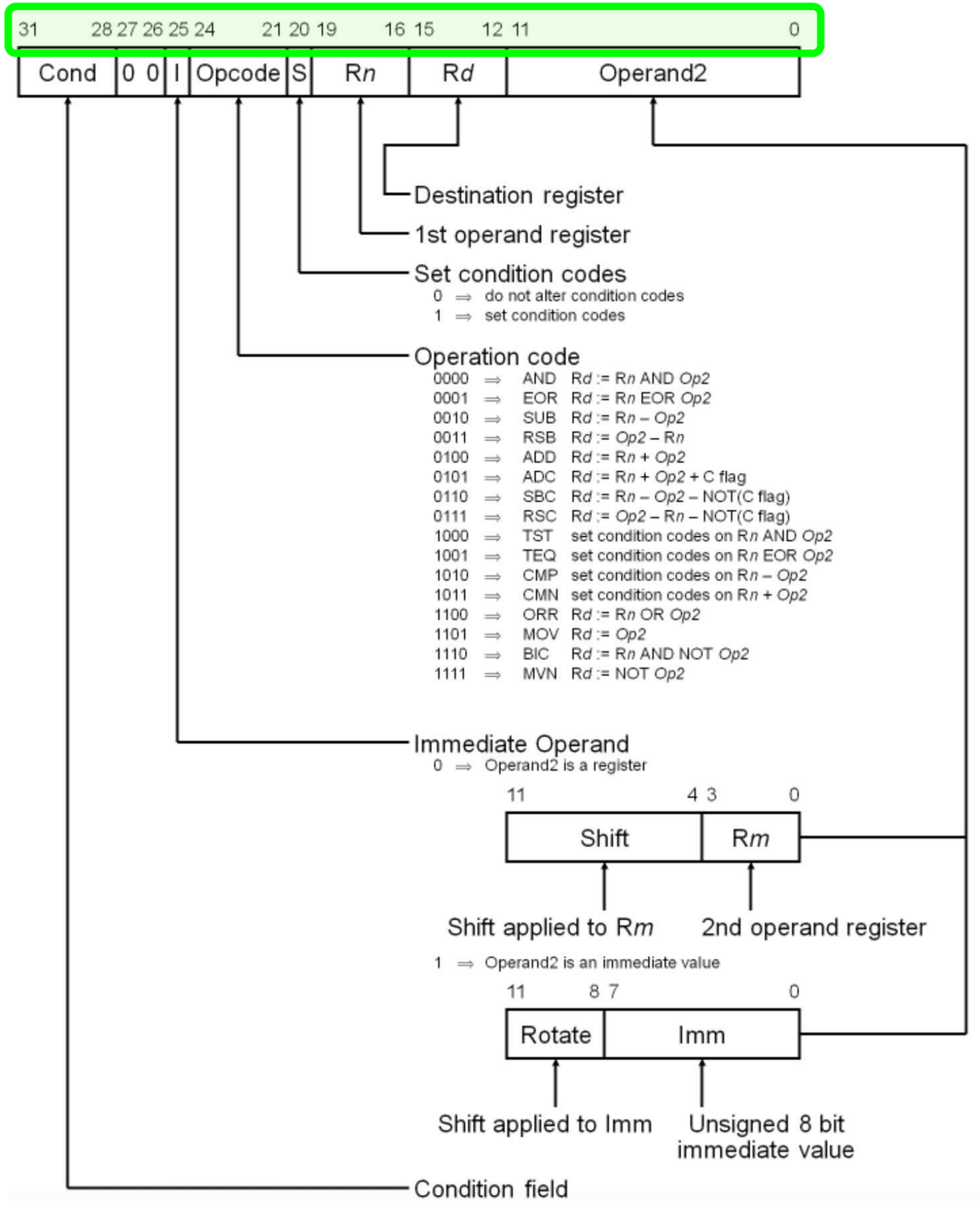
ARM: All bits



- *Find the mapping diagram on MyCourses →*
- *We will see it again shortly.*

Bits	Purpose
31–28	**Condition code (`cond`)**
27–25	Instruction class (data-processing)
24–21	**Opcode**
20	Set condition flags? (`S`)
19–16	**Destination register (`Rd`)**
15–12	Operand register (`Rn`) — unused in MOV
11–0	**Immediate value (rotated)**

See MyCourses → Unit-3 → Cheat Sheet table (32-bit ARM Instructions)



Refresher: Hexadecimal to Binary



Is $1110_2 = 14_{10} = E_{16}$?

1110 → 0xE

- `0xde0` in hexadecimal is:

$$d \times 16^2 + e \times 16^1 + 0 \times 16^0$$

$$d = 13, e = 14 \quad \text{(hexadecimal digits)}$$

$$0xde0 = 13 \times 256 + 14 \times 16 + 0 = 3328 + 224 + 0 = 3552 \,\text{(decimal)}$$

- `0xdc8` in hexadecimal is:

$$d \times 16^2 + c \times 16^1 + 8 \times 16^0$$

$$c = 12, d = 13, 8 = 8 \quad \text{(hexadecimal digits)}$$

$$0xdc8 = 13 \times 256 + 12 \times 16 + 8 = 3328 + 192 + 8 = 3528 \,\text{(decimal)}$$

PART F.1

Programming Assembly: for ARM Instruction Set Arch.

(ii) Understanding Instruction Parts

Writing assembly across instruction sets to see ISAs in action.

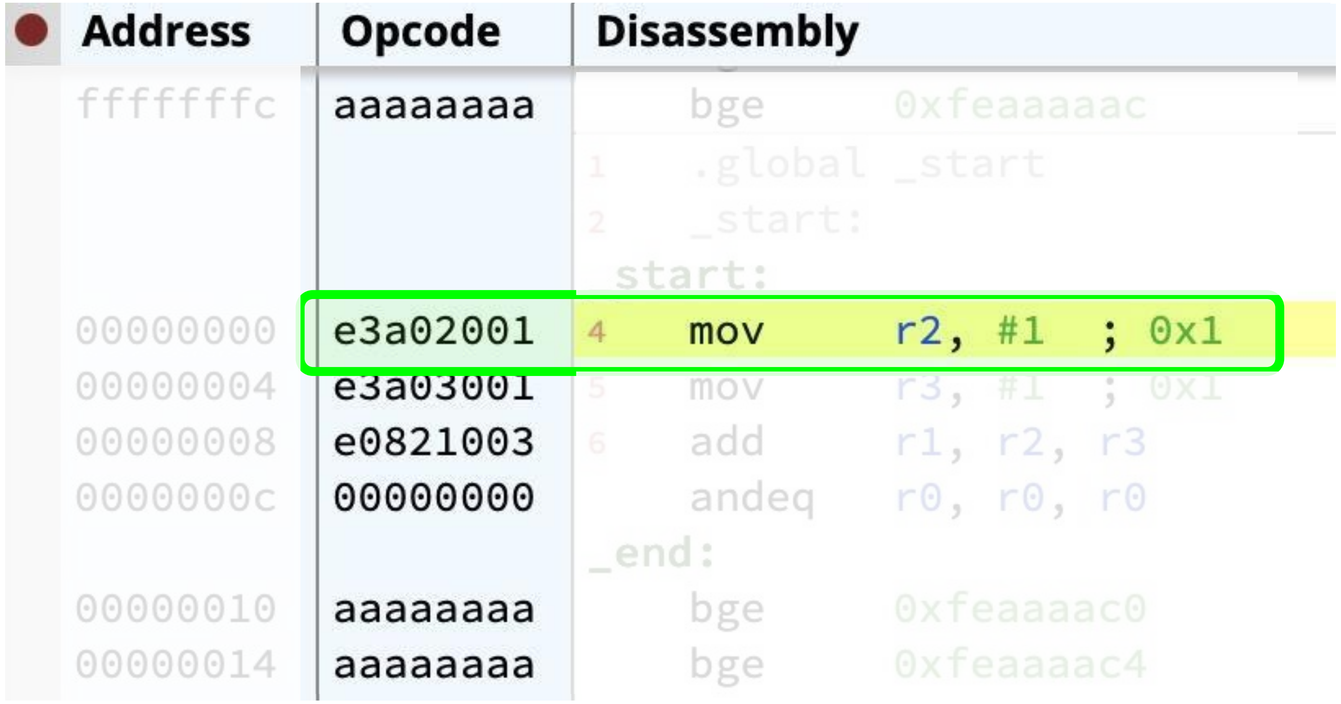
Acorn RISC Machine (ARM)

- **00000000, 00000004, etc. are memory addresses** where each instruction is stored.
- ARM instructions are 4 bytes long, so the **address increases by 4** each time.



Acorn RISC Machine (ARM) : Opcodes

- These steps walk through:
  - Instruction fetch (Fetch & Decode)
  - Execution
  - Register update



ARM: Decoding Instructions

- Step-by-step decoding (**ARMv7** ARM instruction format)
- ARM data-processing immediate format (for **mov** with an **immediate constant**) is:
  1. **Convert from Hex into Bits (in batches of 4-bits, but watch for the split Hex!):**

E3 A0 20 01 → in binary: **1110** 0011 1010 **0000** 0010 **0000 0000 0001**

Bits	Purpose
31–28	Condition code (`cond`)
27–25	Instruction class (data-processing)
24–21	Opcode
20	Set condition flags? (`S`)
19–16	Destination register (`Rd`)
15–12	Operand register (`Rn`) — unused in MOV
11–0	Immediate value (rotated)



ARM: Decoding Instructions

- Step-by-step decoding (**ARMv7** ARM instruction format)

- ARM data-processing immediate format (for **mov** with an **immediate constant**) is:
  1. Break into bytes:

E3 A0 20 01 in binary: **1110** 0011 1010 **0000** 0010 **0000 0000 0001**

Bits	Purpose
31–28	Condition code (`cond`)
27–25	Instruction class (data-processing)
24–21	Opcode
20	Set condition flags? (`S`)
19–16	**Destination register (`Rd`)**
15–12	Operand register (`Rn`) — unused in MOV
11–0	**Immediate value (rotated)**

Address	Opcode	Disassembly
fffffffc	aaaaaaaa	bge 0xfeaaaaac
		1 .global _start
		2 _start:
		_start:
00000000	e3a02001	4 mov r2, #1 ; 0x1
00000004	e3a03001	5 mov r3, #1 ; 0x1
00000008	e0821003	6 add r1, r2, r3
0000000c	00000000	andeq r0, r0, r0
		_end:
00000010	aaaaaaaa	bge 0xfeaaaac0
00000014	aaaaaaaa	bge 0xfeaaaac4

ARM: Decoding Instructions



Watch out for the split Hex!

- Step-by-step decoding (**ARMv7** ARM instruction format)

- ARM data-processing immediate format (for **mov** with an **immediate constant**) is:
  1. Break into bytes:

E3 **A**0 2**0** **01** in binary: **1110** 0011 1010 **0000** 0010 **0000 0000 0001**

Bits	Purpose
31–28	Condition code (`cond`)
27–25	Instruction class (data-processing)
24–21	**Opcode**
20	Set condition flags? (`S`)
19–16	Destination register (`Rd`)
15–12	Operand register (`Rn`) — unused in MOV
11–0	Immediate value (rotated)

Address	Opcode	Disassembly
fffffffc	aaaaaaaa	bge 0xfeaaaaac
		1 .global _start
		2 _start:
		_start:
00000000	e3a02001	4 mov r2, #1 ; 0x1
00000004	e3a03001	5 mov r3, #1 ; 0x1
00000008	e0821003	6 add r1, r2, r3
0000000c	00000000	andeq r0, r0, r0
		_end:
00000010	aaaaaaaa	bge 0xfeaaaac0
00000014	aaaaaaaa	bge 0xfeaaaac4

ARM: Decoding Instructions

- Step-by-step decoding (**ARMv7** ARM instruction format)

- ARM data-processing immediate format (for **mov** with an **immediate constant**) is:
  1. Break into bytes:

  E3 **A0** 2**0 01** in binary: **1110** 0011 1010 **0000** 0010 **0000 0000 0001**

Bits	Purpose
31–28	**Condition code (`cond`)**
27–25	Instruction class (data-processing)
24–21	**Opcode**
20	Set condition flags? (`S`)
19–16	**Destination register (`Rd`)**
15–12	Operand register (`Rn`) — unused in MOV
11–0	**Immediate value (rotated)**

Address	Opcode	Disassembly
fffffffc	aaaaaaaa	bge 0xfeaaaaac
		1 .global _start
		2 _start:
		_start:
00000000	e3a02001	4 mov r2, #1 ; 0x1
00000004	e3a03001	5 mov r3, #1 ; 0x1
00000008	e0821003	6 add r1, r2, r3
0000000c	00000000	andeq r0, r0, r0
		_end:
00000010	aaaaaaaa	bge 0xfeaaaac0
00000014	aaaaaaaa	bge 0xfeaaaac4

ARM: 31:28
Condition Codes

- Step-by-step decoding (**ARMv7** ARM instruction format)

- ARM data-processing immediate format (for **mov** with an **immediate constant**) is:

1. Break into bytes:

E3 **A0** 2**0** **01** in binary: **1110** 0011 1010 **0000** 0010 **0000 0000 0001**

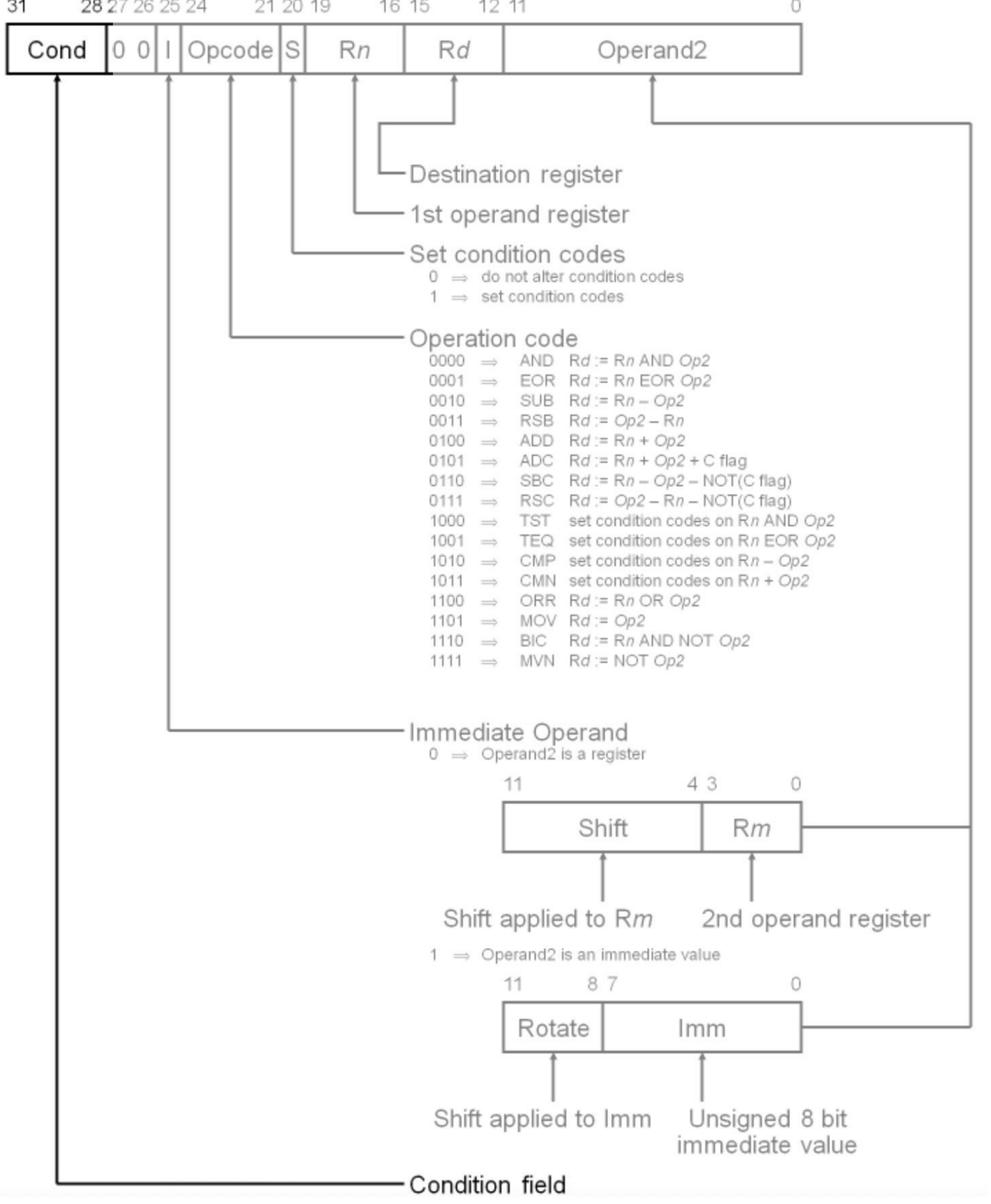
```
e3a02001    4    mov     r2, #1  ; 0x1
```

2. Decode **Condition Code** (bits 31–28)

1110 → 0xE → **AL** = Always execute

ARM: 31:28
Condition Codes

See MyCourses → Unit-3 → Cheat Sheet table (32-bit ARM Instructions)



ARM: 31:28
Condition Codes

2. Decode **Condition Code** (bits 31–28)
   1110 → 0xE → **AL** = Always execute

See MyCourses → Unit-3 → Cheat Sheet table (Condition Codes)

Table A3-1 Condition codes

Opcode [31:28]	Mnemonic extension	Meaning	Condition flag state
0000	EQ	Equal	Z set
0001	NE	Not equal	Z clear
0010	CS/HS	Carry set/unsigned higher or same	C set
0011	CC/LO	Carry clear/unsigned lower	C clear
0100	MI	Minus/negative	N set
0101	PL	Plus/positive or zero	N clear
0110	VS	Overflow	V set
0111	VC	No overflow	V clear
1000	HI	Unsigned higher	C set and Z clear
1001	LS	Unsigned lower or same	C clear or Z set
1010	GE	Signed greater than or equal	N set and V set, or N clear and V clear (N == V)
1011	LT	Signed less than	N set and V clear, or N clear and V set (N != V)
1100	GT	Signed greater than	Z clear, and either N set and V set, or N clear and V clear (Z == 0,N == V)
1101	LE	Signed less than or equal	Z set, or N set and V clear, or N clear and V set (Z == 1 or N != V)
1110	AL	Always (unconditional)	-
1111	-	See *Condition code 0b1111*	-

ARM: 31:28
Condition Codes

See MyCourses → Unit-3 → Cheat Sheet table (Condition Codes)

31-28	27-26	25	24-21	20	19-16	15-12	11-0
cond	00	I	opcode	S	Rn	Rd	Operand2
1110	00	1	1101	0	0000	0010	000000000001
e	3		a		0	2	0 0 1

```
mov r2, #1  ; 0x1
```

Address	Opcode	Disassembly
fffffffc	aaaaaaaa	bge 0xfeaaaaac
		1 .global _start
		2 _start:
		_start:
00000000	e3a02001	4 mov r2, #1 ; 0x1
00000004	e3a03001	5 mov r3, #1 ; 0x1
00000008	e0821003	6 add r1, r2, r3
0000000c	00000000	andeq r0, r0, r0
		_end:
00000010	aaaaaaaa	bge 0xfeaaaac0
00000014	aaaaaaaa	bge 0xfeaaaac4

Table A3-1 Condition codes

Opcode [31:28]	Mnemonic extension	Meaning	Condition flag state
1000	HI	Unsigned higher	C set and Z clear
1001	LS	Unsigned lower or same	C clear or Z set
1010	GE	Signed greater than or equal	N set and V set, or N clear and V clear (N == V)
1011	LT	Signed less than	N set and V clear, or N clear and V set (N != V)
1100	GT	Signed greater than	Z clear, and either N set and V set, or N clear and V clear (Z == 0,N == V)
1101	LE	Signed less than or equal	Z set, or N set and V clear, or N clear and V set (Z == 1 or N != V)
1110	AL	Always (unconditional)	-
1111	-	See *Condition code 0b1111*	-

ARM: 31:28
Condition Codes

See MyCourses → Unit-3 → Cheat Sheet table (Condition Codes)

31-28	27-26	25	24-21	20	19-16	15-12	11-0
cond	00	I	opcode	S	Rn	Rd	Operand2
1110	00	0	0100	0	0010	0001	000000000011
e	0		8		2	1	0 0 3
add r1, r2, r3					r2	r1	r3

Address	Opcode	Disassembly
fffffffc	aaaaaaaa	bge 0xfeaaaaac
		1 .global _start
		2 _start:
		_start:
00000000	e3a02001	4 mov r2, #1 ; 0x1
00000004	e3a03001	5 mov r3, #1 ; 0x1
00000008	e0821003	6 add r1, r2, r3
0000000c	00000000	andeq r0, r0, r0
		_end:
00000010	aaaaaaaa	bge 0xfeaaaac0
00000014	aaaaaaaa	bge 0xfeaaaac4

Table A3-1 Condition codes

Opcode [31:28]	Mnemonic extension	Meaning	Condition flag state
1000	HI	Unsigned higher	C set and Z clear
1001	LS	Unsigned lower or same	C clear or Z set
1010	GE	Signed greater than or equal	N set and V set, or N clear and V clear (N == V)
1011	LT	Signed less than	N set and V clear, or N clear and V set (N != V)
1100	GT	Signed greater than	Z clear, and either N set and V set, or N clear and V clear (Z == 0,N == V)
1101	LE	Signed less than or equal	Z set, or N set and V clear, or N clear and V set (Z == 1 or N != V)
1110	AL	Always (unconditional)	-
1111	-	See *Condition code 0b1111*	-

ARM: 24:21
Operation Codes

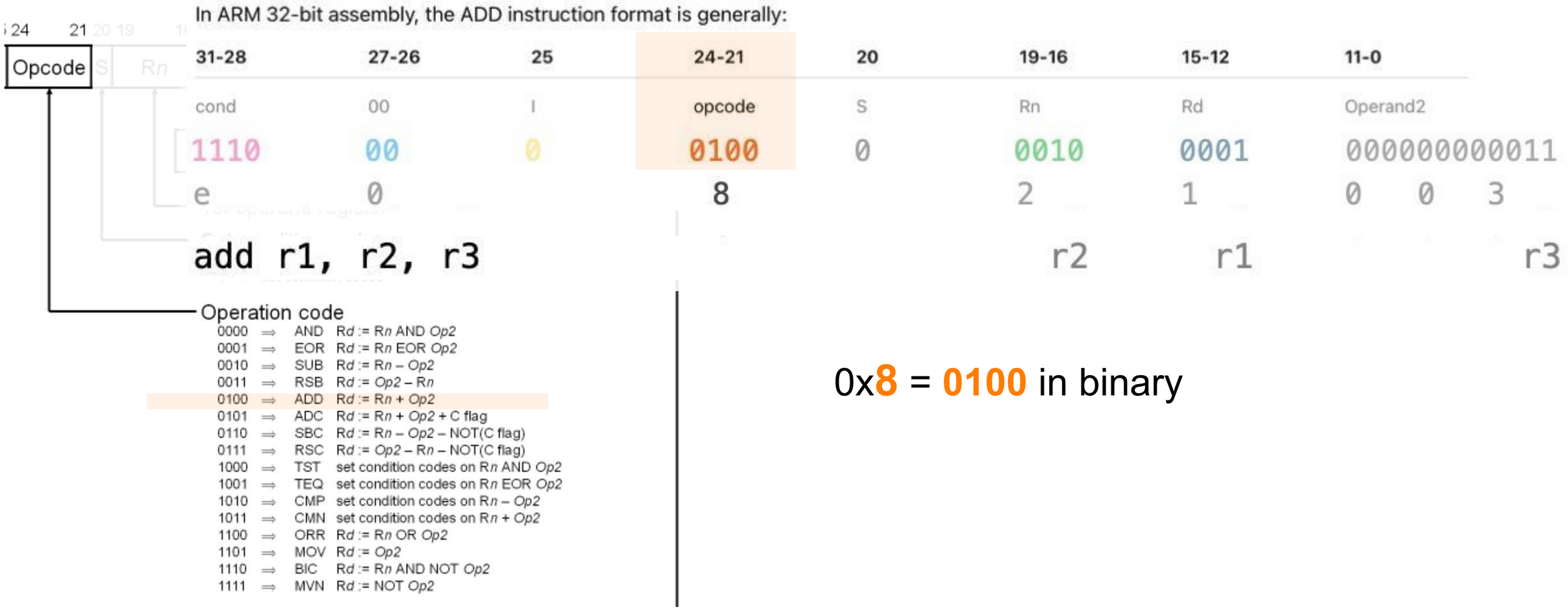


See MyCourses → Unit-3 → Cheat Sheet table (32-bit ARM Instructions)

Watch out for the split Hex!

ARM: 24:21
Operation Codes

See MyCourses → Unit-3 → Cheat Sheet table (32-bit ARM Instructions)



0x**8** = **0100** in binary

ARM: 24:21 Operation Codes

See MyCourses → Unit-3 → Cheat Sheet table (32-bit ARM Instructions)

In ARM 32-bit assembly, the ADD instruction format is generally:

31-28	27-26	25	24-21	20	19-16	15-12	11-0
cond	00	I	opcode	S	Rn	Rd	Operand2
1110	00	1	1101	0	0000	0010	000000000001
e	3		a		0	2	0 0 1

```
mov r2, #1  ; 0x1
```

Opcode

Operation code

0000	⇒	AND	Rd := Rn AND Op2
0001	⇒	EOR	Rd := Rn EOR Op2
0010	⇒	SUB	Rd := Rn – Op2
0011	⇒	RSB	Rd := Op2 – Rn
0100	⇒	ADD	Rd := Rn + Op2
0101	⇒	ADC	Rd := Rn + Op2 + C flag
0110	⇒	SBC	Rd := Rn – Op2 – NOT(C flag)
0111	⇒	RSC	Rd := Op2 – Rn – NOT(C flag)
1000	⇒	TST	set condition codes on Rn AND Op2
1001	⇒	TEQ	set condition codes on Rn EOR Op2
1010	⇒	CMP	set condition codes on Rn – Op2
1011	⇒	CMN	set condition codes on Rn + Op2
1100	⇒	ORR	Rd := Rn OR Op2
1101	⇒	MOV	Rd := Op2
1110	⇒	BIC	Rd := Rn AND NOT Op2
1111	⇒	MVN	Rd := NOT Op2

0x**a** = **1101** in binary

ARM: Decoding Instructions: Other bits

- ARM data-processing immediate format (for mov with an immediate constant) is:
  1. Break into bytes:

E3 A0 20 01 in binary: 1110 0011 1010 0000 0010 0000 0000 0001

```
e3a02001    4    mov     r2, #1  ; 0x1
```

3. Bits 27–25 (Instruction class)
   001 → Data-processing immediate
4. Bits 24–21 (**Opcode**)
   1101 → **MOV opcode** (in ARM data-processing instructions)
5. Bit 20 (S flag)
   0 → MOV Do not update **CPSR** flags
6. Bits 19–16 (Rn)
   0000 → R0 (unused for MOV immediate)

ARM: Decoding Instructions: Other bits

- ARM data-processing immediate format (for mov with an immediate constant) is:

1. Break into bytes:

E3 A0 20 01 in binary: 1110 0011 1010 0000 0010 0000 0000 0001

```
e3a02001    4    mov     r2, #1  ; 0x1
```

7. Bits 15–12 (Destination register Rd)

   0010 → R2

8. Bits 11–0 (Immediate + rotate)

   0000 0000 0001 → Immediate value 1

   (rotation = 0, so no shift applied)

PART F.1

Programming Assembly: for ARM Instruction Set Arch.

*(iii) Understanding **Move** In Detail*

Writing assembly across instruction sets to see ISAs in action.

ARM: Decoding Instructions: Move in Detail

- This machine code means:
  - Execute unconditionally
  - Perform **MOV**
    - Rd = R2
    - Immediate = 1

Address	Opcode	Disassembly
fffffffc	aaaaaaaa	bge 0xfeaaaaac
		1 .global _start
		2 _start:
		_start:
00000000	e3a02001	4 mov r2, #1 ; 0x1
00000004	e3a03001	5 mov r3, #1 ; 0x1
00000008	e0821003	6 add r1, r2, r3
0000000c	00000000	andeq r0, r0, r0
		_end:
00000010	aaaaaaaa	bge 0xfeaaaac0
00000014	aaaaaaaa	bge 0xfeaaaac4

ARM: Decoding Instructions: Move in Detail

- MOV Command
  - MOV = Move (really “copy”)
  - It copies a value into a register, without changing the source.
  - It does not move in the “cut-and-paste” sense — the original stays where it was.
    MOV <destination>, <source>

  - Where:
    - <destination> = register to store the result (e.g., R1)
    - <source> = register or immediate value (e.g., R3 or #7)

ARM: Decoding Instructions: Move in Detail

- Examples
  - Move an immediate value into a register

    MOV R2, #5 ; R2 = 5

  - Immediate values use #.
  - After execution, R2 contains decimal 5 (0x00000005 in hex).

ARM: Decoding Instructions: Move in Detail

- Examples
  - Move value from one register to another

    MOV R2, R1 ; R1 = R2

  - Copy whatever is in R2 into R1.
  - Both registers now hold the same value, but they're independent.

ARM: Decoding Instructions: Move in Detail

- Examples
  - Move with flags (MOVS)

    MOVS R1, #0 ; R1 = 0 and update status flags

    - MOVS sets the **Zero flag (Z=1)** in the **CPSR** because the result is **zero**.

ARM: Decoding Instructions: Move in Detail

- In the CPU
  - When we do

    MOVS R2, #5 ; R2 = 5 and update status flags

    1. **Instruction Fetch** — CPU fetches 0x**E3A02005** from memory.
    2. Decode — CPU sees it's a MOV, immediate, destination=R2, value=5.
    3. Execute — **ALU** takes the immediate 5, writes it to R2.
    4. **Write Back** — **Register file** stores 5 in

ARM: Decoding Instructions: Add Example

- Examples
  - Add the values in registers R2 and R3, then store the result in register R1.

    ADD R1, R2, R3 ; R1 = 0 and update status flags

    ADD <destination>, <source1>, <source2>

  - Where:
    - <destination> = register to store the result (e.g., R1)
    - <source1> = register to store the value of the operand 1
    - <source2> = register to store the value of the operand 2



Key Points: ISAs with Assembly (ARM)

- **ARM is a RISC architecture used in mobile/embedded systems**
  - Uses load-store model with 16 general-purpose registers
  - Condition codes on every instruction (e.g. MOVS sets flags)
- **ARM instruction encoding uses 32-bit fixed format**
  - Condition field (bits 31:28), opcode field, register fields
- **Key ARM instructions: MOV, ADD for register operations**

PART F.1.2

Programming Assembly: ARM

Example Instructions: MOV & ADD Coding Practice

Writing assembly across instruction sets to see ISAs in action.

ARM: add

- Examples
  - Add the values in registers R2 and R3, then store the result in register R1.

    ADD R1, R2, R3 ; R1 = 0 and update status flags

E0 82 10 03 in binary: 1110 0000 1000 0010 0001 0000 0000 0011

In ARM 32-bit assembly, the ADD instruction format is generally:

31-28	27-26	25	24-21	20	19-16	15-12	11-0
cond	00	I	opcode	S	Rn	Rd	Operand2

Acorn RISC Machine (ARM)

- Examples
  - Add the values in registers R2 and R3, then store the result in register R1.

```
ADD R1, R2, R3   ; R1 = 0 and update status flags
```

E0 82 10 03 in binary: 1110 0000 1000 0010 0001 0000 0000 0011

In ARM 32-bit assembly, the ADD instruction format is generally:

31-28	27-26	25	24-21	20	19-16	15-12	11-0
cond	00	I	opcode	S	Rn	Rd	Operand2

- cond (4bits) = 1110 (binary for e hex) → Always
- I (1 bit) = 0 (immediate (I) or register operand)
- **opcode** (4 bits) = 0100 (4 decimal) → **ADD** instruction

Acorn RISC Machine (ARM)

- Examples
  - Add the values in registers R2 and R3, then store the result in register R1.

ADD R1, R2, R3 ; R1 = 0 and update status flags

E0 82 10 03 in binary: 1110 0000 1000 0010 0001 0000 0000 0011

In ARM 32-bit assembly, the ADD instruction format is generally:

31-28	27-26	25	24-21	20	19-16	15-12	11-0
cond	00	I	opcode	S	Rn	Rd	Operand2

- S (1 bit) = 0 (do not update condition flags)
- Rn (4 bits) = first operand register (R2)
- Rd (4 bits) = destination register (R1)
- Operand2 (12 bits) = second operand register (R3) in register form

Acorn RISC Machine (ARM)

- Examples
  - Add the values in registers R2 and R3, then store the result in register R1.

ADD R1, R2, R3 ; R1 = 0 and update status flags

E0 82 10 03 in binary: 1110 0000 1000 0010 0001 0000 0000 0011

In ARM 32-bit assembly, the ADD instruction format is generally:

31-28	27-26	25	24-21	20	19-16	15-12	11-0
cond	00	I	opcode	S	Rn	Rd	Operand2

- Register mapping:
  - Rn = 2 → R2 (first source)
  - Rd = 1 → R1 (destination)
  - Operand2 = 3 → R3 (second source)

EXERCISE

Exercise 2.1: ALU - ARM with Opcode explanation



1. Copy / Rewrite the code from the example screenshot (slides) and find the answers.
2. Write **Assembly code** with **Opcode** explanation for: *[Calculate]*

- **Addition,** when **R2=21** and **R3=12** (add)
- **Subtraction,** when **R2=21** and **R3=12** (sub)
- **Multiplication,** when **R2=21** and **R3=10** (mul)
- **Division,** when **R2=21** and **R3=10** (sdiv) (Integer Division)

3. Save your code into a text file for your later reference – e.g. **ARM-32bit_Add.txt**

https://cpulator.01xz.net/?sys=arm



Key Points: Programming: ISAs with Assembly

- **ARM Assembly**
  - MOV, ADD, SUB instructions
  - Machine code decoding (ARMv7 format)
  - Condition codes (bits 31–28)
- **Instruction Format**
  - Opcode + Operand structure
  - Memory-reference, register-reference, I/O types
- **Architecture Selection**
  - ARM (mobile),
  - MIPS (education/embedded systems)
  - x86 (laptop/ desktop/ server)

PART F.2

Programming Assembly: for MIPS Instruction Set Arch

Writing assembly across instruction sets to see ISAs in action.

MIPS

- MIPS stands for Microprocessor without Interlocked **Pipeline** Stages
- Full name: Microprocessor without Interlocked Pipeline Stages
- Origin: Developed by MIPS Computer Systems in the 1980s.
- Instruction format:
  - Fixed 32-bit instructions,
  - Simple **3-operand** style (**ADD** R1, R2, R3).
- Use in education:
  - Extremely popular in teaching computer architecture because it's **clean** and **easy** to read.
  - Tools like SPIM, MARS simulate it.

MIPS Registers:

- 32 general-purpose registers.

Result value registers:
- Named **$v0, $v1**

Argument registers:
- Named **$a0 – $a3**

Temporary registers:
- Named **$t0, $t1 — $t9**

… and a few more.

Variable registers:
- Named **$s0, $s1 — $s7**

#	Name	Notes
0	zero	always contains 0
1	at	reserved for assembler
2	v0	expression evaluation and function results
3	v1	
4	a0	arguments passed to procedures and functions
5	a1	
6	a2	
7	a3	
8	t0	temporary storage – can be overwritten by called routines
…	…	
15	t7	

#	Name	Notes
16	s0	variable storage – must be saved by called routines
…	…	
23	s7	
24	t8	more temporary storage
25	t9	
26	k0	reserved for OS use
27	k1	
28	gp	global area pointer
29	sp	stack pointer
30	fp	frame pointer
31	ra	return address register

Source: https://www.cs.rit.edu/~paw/250/notes/135e/,notes.221/02/MIPS%20registers.pdf

We focus on MIPS in this subject, so you will meet all of these registers.

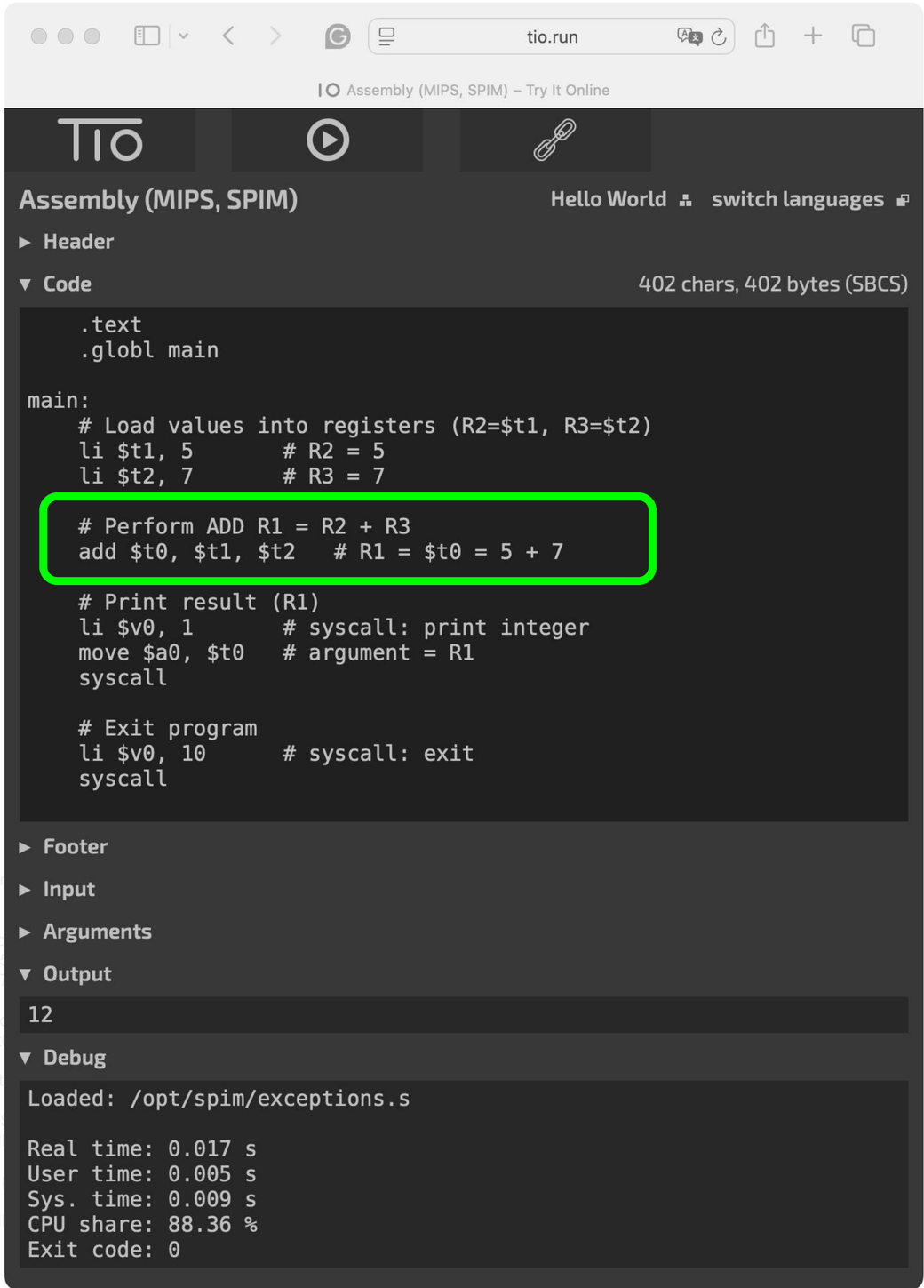
MIPS

- MIPS uses instructions like **ADD** R1, R2, R3 directly.

- This way:
  - $t0 acts as R1
  - $t1 acts as R2
  - $t2 acts as R3

- We can change numbers in li $t1, ... and li $t2, ... to test.

- https://tio.run/# then choose the language "Assembly (MIPS, SPIM)"

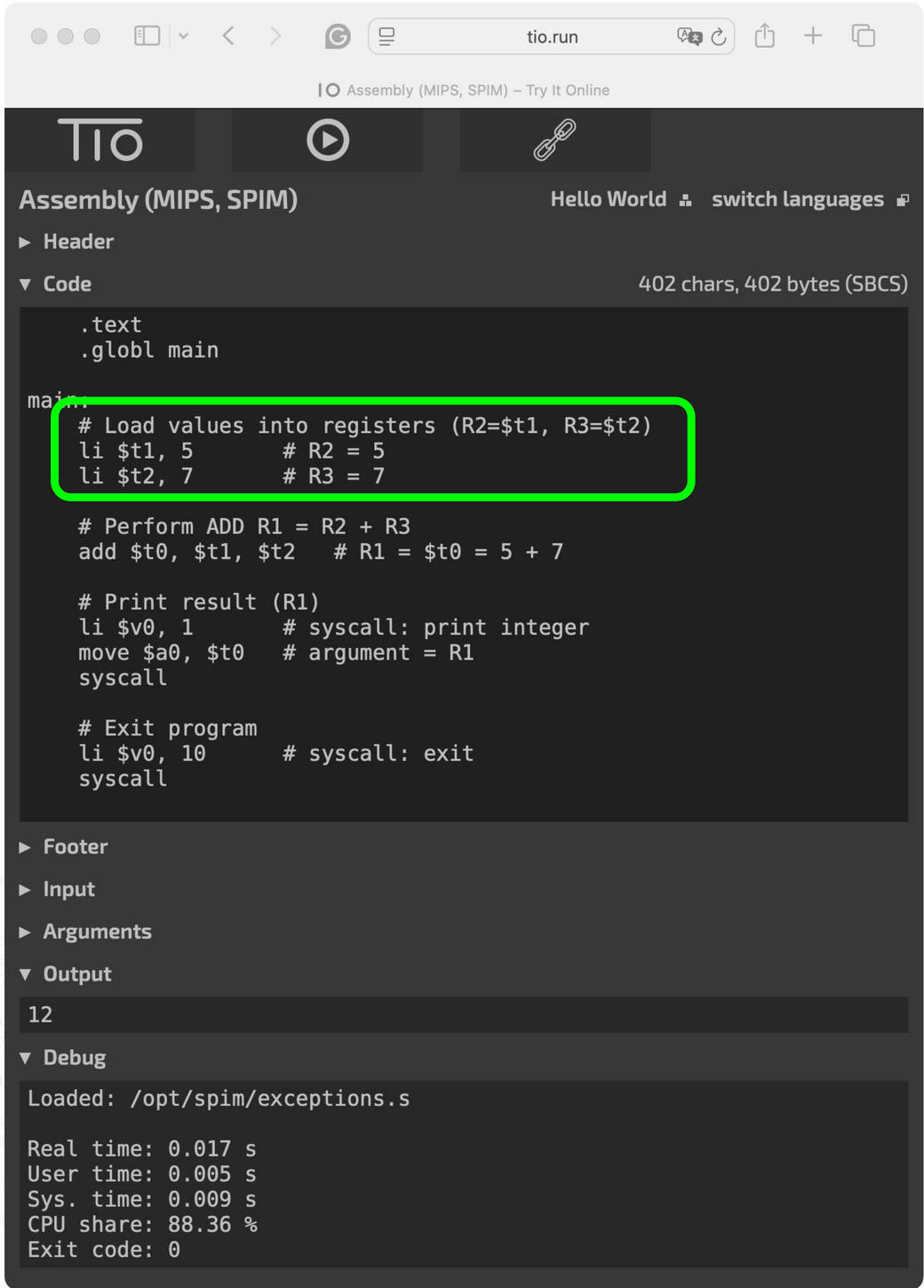


MIPS

- Initialization (Register Loading)

```
li $t1, 5       # R2 = $t1 = 5
li $t2, 7       # R3 = $t2 = 7
```

- li (Load Immediate): Loads a constant value into a register.
- $t1 (temporary register 1) is set to 5.
- $t2 (temporary register 2) is set to 7.
- Note:
  - In the comments, R2 and R3 refer to the logical registers in your original question (mapped to $t1 and $t2 in MIPS).

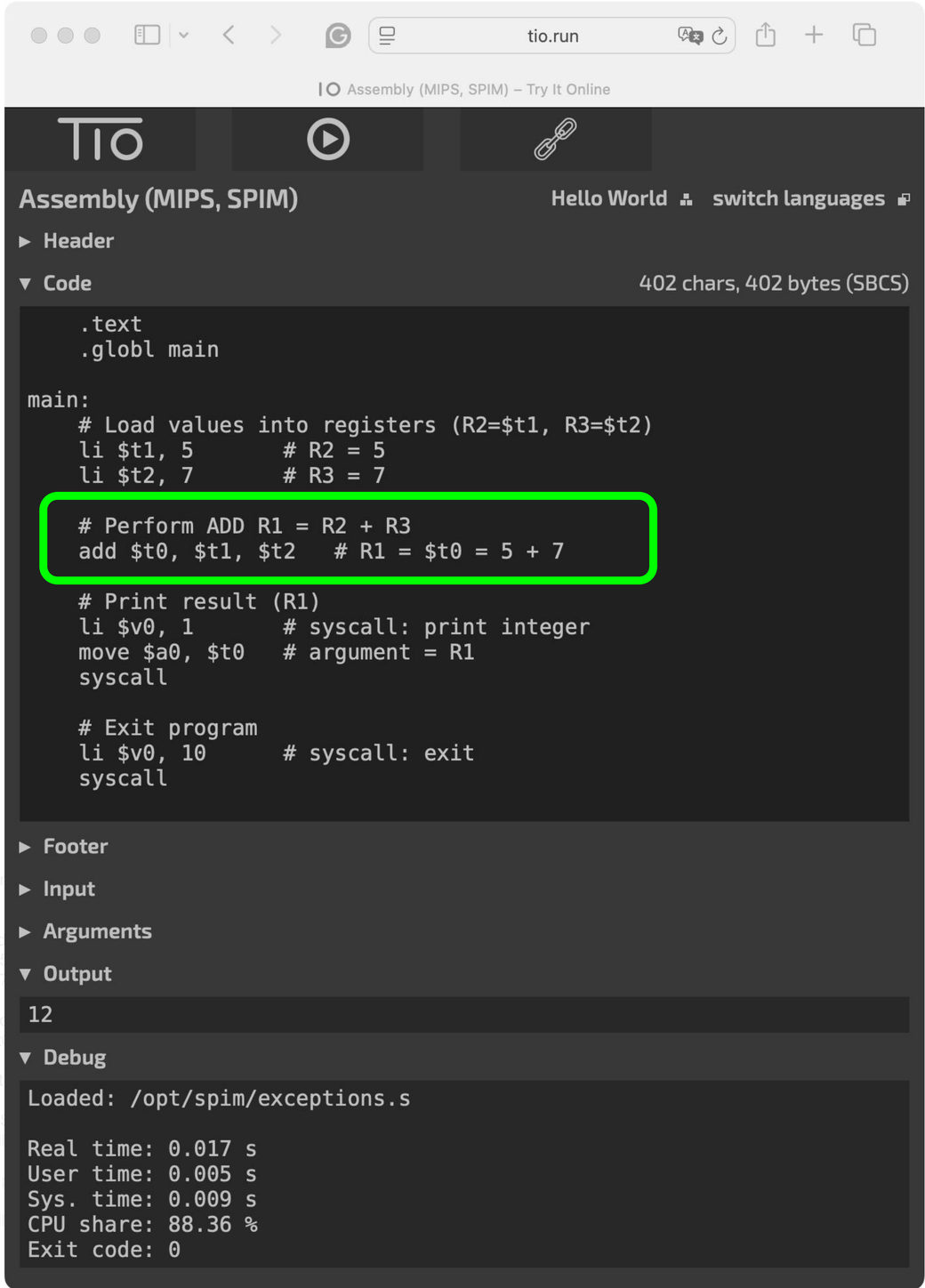


MIPS

- Addition Operation

  add $t0, $t1, $t2 # R1 = $t0 = 5 + 7

- add: Adds the values of $t1 (5) and $t2 (7), storing the result in $t0 (temporary register 0).
- Result: $t0 = 12 (which is referred to as R1 in the comments).

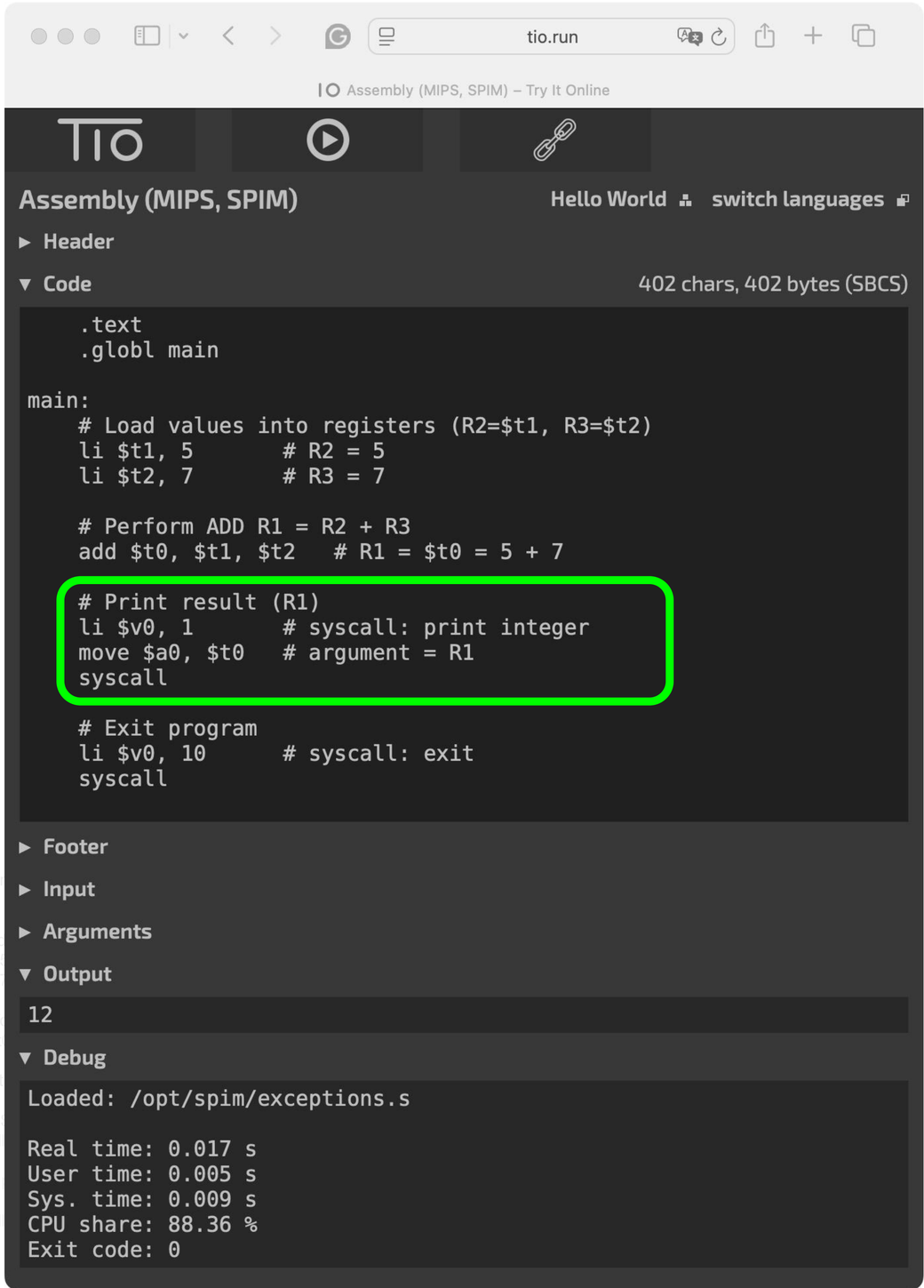


MIPS

- Print the Result

```
li $v0, 1          # Prepare to print an integer
move $a0, $t0      # Set argument = $t0 (R1 = 12)
syscall            # Execute the print
```

- li $v0, 1: Loads the syscall code 1 into $v0 (MIPS uses $v0 to determine the syscall type).
  - 1 = “print integer” service.
- move $a0, $t0: Copies the value of $t0 (12) to $a0 (the register for syscall arguments).
- syscall: Triggers the syscall, printing the integer (12) to the console.

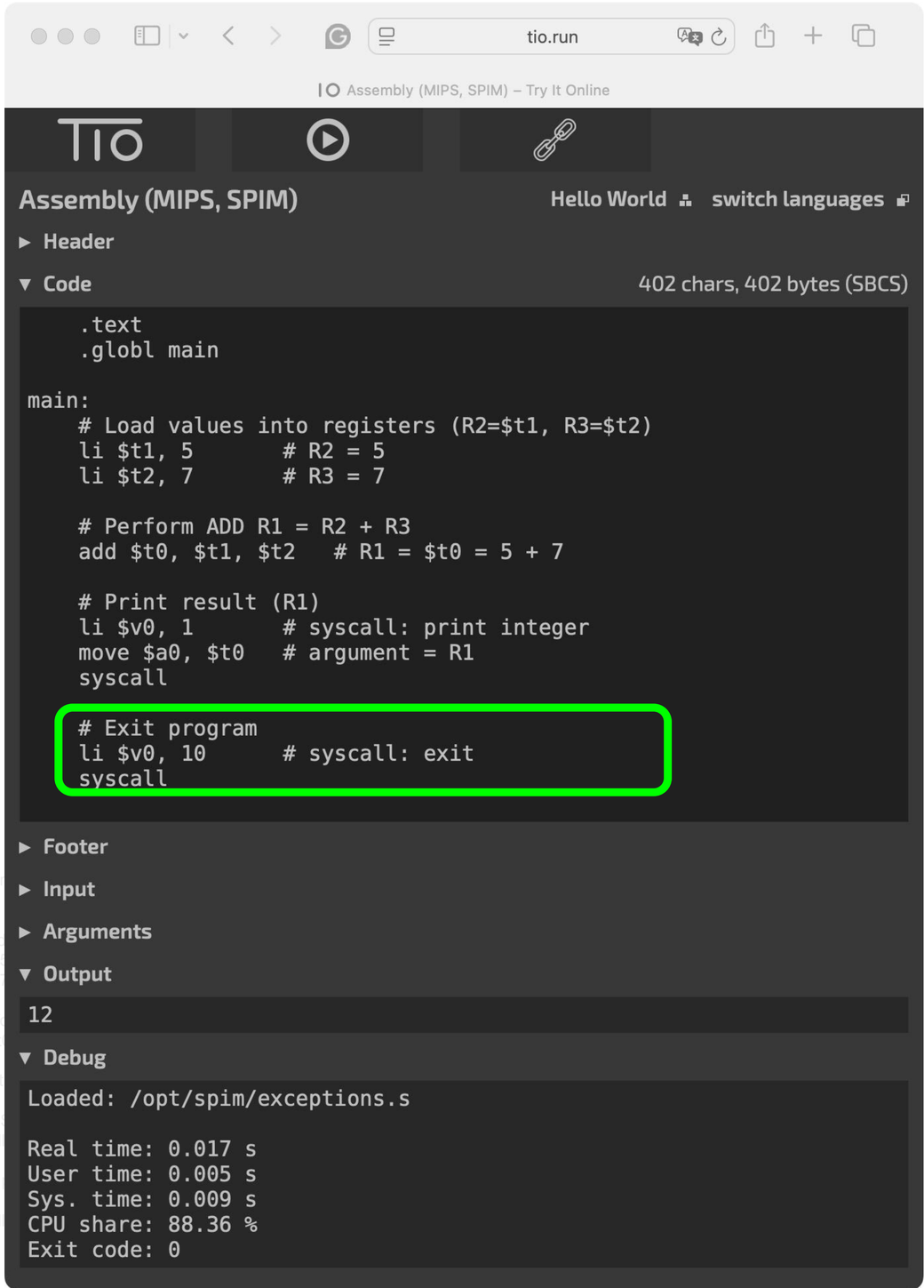


MIPS

- Exit the Program

```
li $v0, 10      # Prepare to exit
syscall         # Terminate the program
```

- li $v0, 10: Loads the syscall code 10 into $v0.
  - 10 = “exit” service.
- syscall: Ends the program execution.



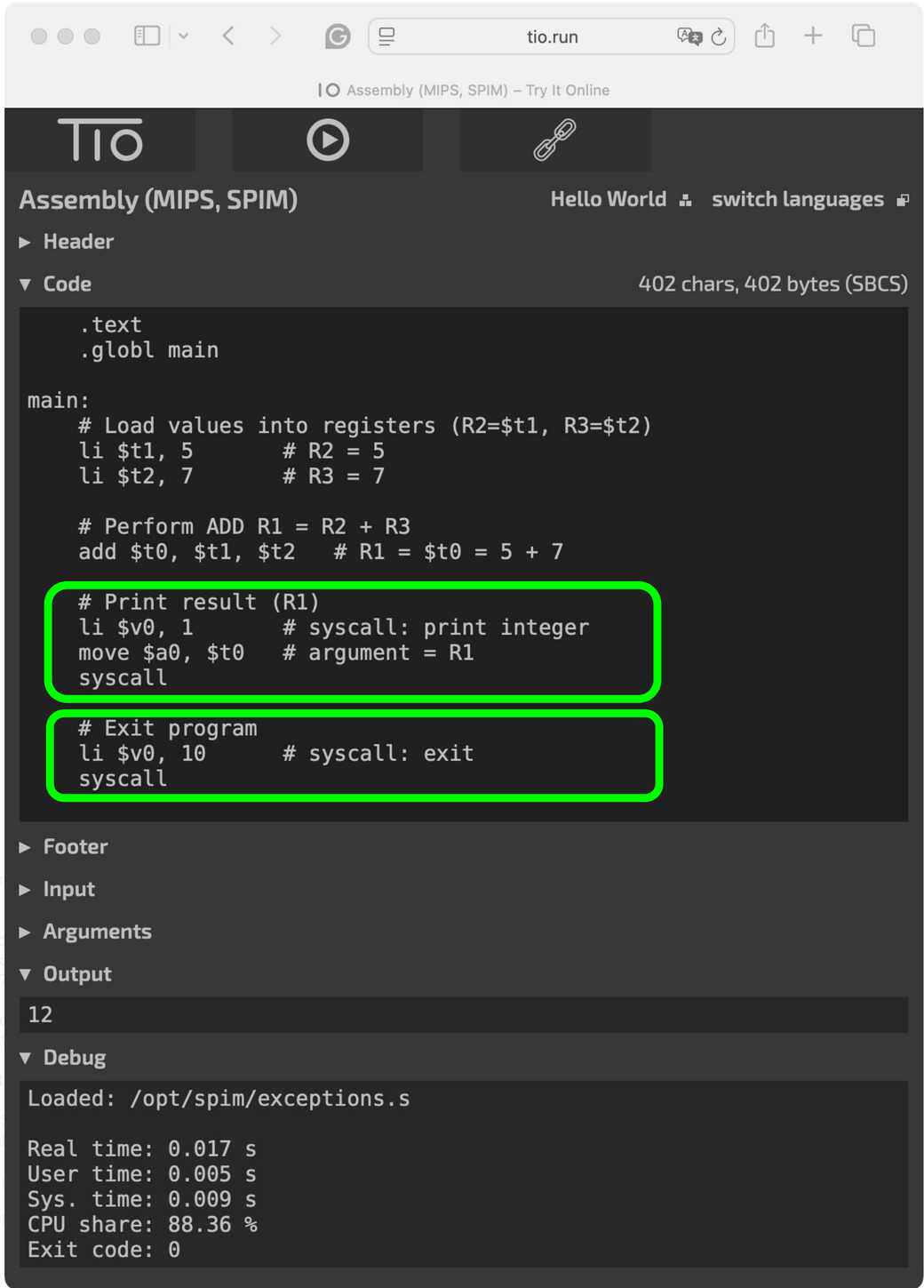
MIPS

MIPS Register Conventions:

- **$t0–$t9**: Temporary registers (caller-saved).
- **$v0**: Holds syscall numbers and return values.
- **$a0–$a3**: Store arguments for syscalls/ functions.

Syscalls:

- MIPS uses **syscall** to interact with the OS, e.g.,
  - Program termination = **10**
  - Print Integer = **1**
  - I/O, = ….



Code to copy/paste

EXERCISE

Exercise 2.2: ALU - MIPS



1. Copy / Rewrite the code from the example screenshot (slides) and find the answers.
2. Write **Assembly code** for: *[Calculate; Print; Exit]*

- **Addition,** when **R2=21** and **R3=12** (add)
- **Subtraction,** when **R2=21** and **R3=12** (sub)
- **Multiplication,** when **R2=21** and **R3=10** (mul)
- **Division,** when **R2=21** and **R3=10** (div) (Integer Division)

3. Save your code into a text file for your later reference – e.g. **MIPS_Add.txt**

https://tio.run/#spim and/or MARS.jar Simulator (Download via MyCourses)



Key Points: Programming: ISAs with Assembly

- **MIPS Assembly**
  - Registers: $t0–$t9, $v0, $a0
  - System calls: print integer, exit
  - Instructions: li, add, syscall

- **Architecture Selection**
  - ARM (mobile),
  - MIPS (education/embedded systems)
  - x86 (laptop/ desktop/ server)

PART F.3

Programming Assembly: for X86 Instruction Set Arch

Writing assembly across instruction sets to see ISAs in action.

x86 Architecture

- x86 architecture refers to a family of instruction set architectures (**ISA**s) initially developed by Intel, based on the 8086 microprocessor.
- It's a **Complex Instruction Set Computer** (**CISC**) design.
- It uses a wide range of instructions that can perform multiple tasks in one operation.
- x86 is the dominant architecture for personal computers and servers.
- The assembly editor below runs on x86.

```
section .data
   result db 0, 10   ; space for one digit + newline

section .text
   global _start

_start:
   mov eax, 5        ; move 5 into eax register
   add eax, 3        ; eax = 8
   add al, '0'       ; convert to ASCII
   mov [result], al  ; store result

   mov eax, 4        ; sys_write
   mov ebx, 1        ; move 1 into ebx register
   mov ecx, result
   mov edx, 2
   int 0x80

   mov eax, 1
   xor ebx, ebx      ; set ebx to 0.
   int 0x80
```

https://www.tutorialspoint.com/compilers/online-assembly-compiler.htm

Registers

x86

- The **CPU registers:**
  - Small, ultra-fast storage locations inside the processor.

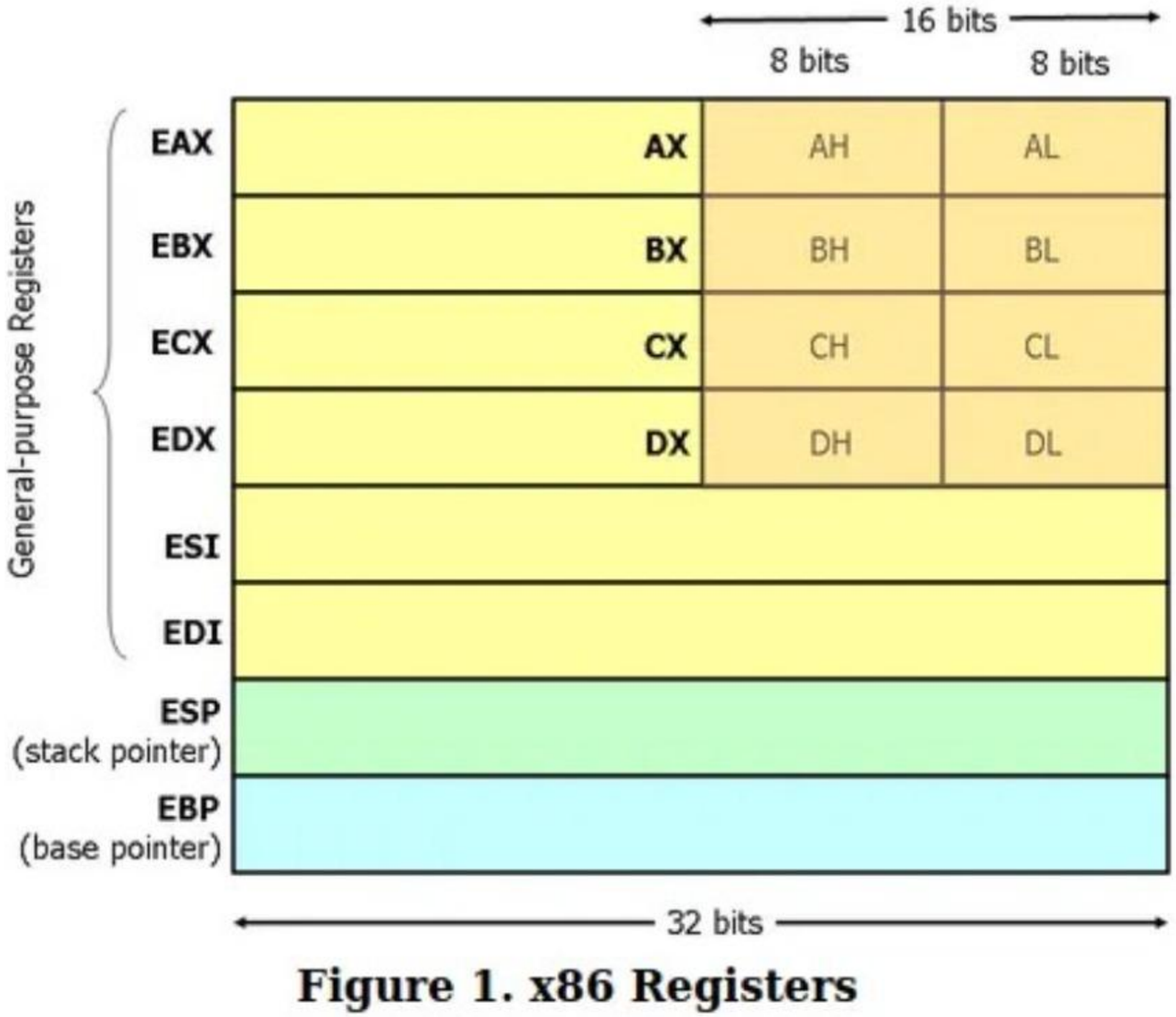


Figure 1. x86 Registers

Registers

x86

- **eax** — “Accumulator Register”
  - Often used to store results of calculations, and in Linux 32-bit syscalls.
  - It holds the system call number, e.g.,
    - 4 for write,
    - 1 for exit.

```
section .data
    result db 0, 10   ; space for one digit + newline

section .text
    global _start

_start:
    mov eax, 5        ; move 5 into eax register
    add eax, 3        ; eax = 8
    add al, '0'       ; convert to ASCII
    mov [result], al  ; store result

    mov eax, 4        ; sys_write
    mov ebx, 1        ; move 1 into ebx register
    mov ecx, result
    mov edx, 2
    int 0x80

    mov eax, 1
    xor ebx, ebx
    int 0x80
```



Registers

x86

- **ebx** — “Base Register”
  - In syscalls, it often holds the first argument to the system call.
  - Example: for writing to the screen, it stores the file descriptor
    - 1 means standard output.

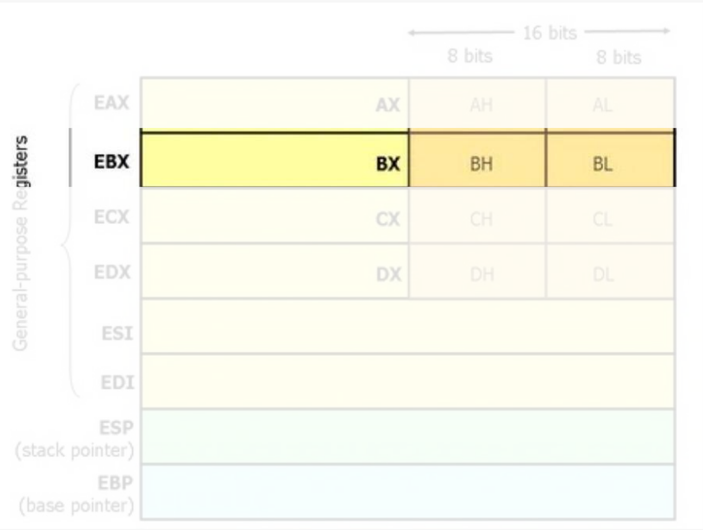
```
section .data
    result db 0, 10   ; space for one digit + newline

section .text
    global _start

_start:
    mov eax, 5        ; move 5 into eax register
    add eax, 3        ; eax = 8
    add al, '0'       ; convert to ASCII
    mov [result], al  ; store result

    mov eax, 4        ; sys_write
    mov ebx, 1        ; move 1 into ebx register
    mov ecx, result
    mov edx, 2
    int 0x80

    mov eax, 1
    xor ebx, ebx
    int 0x80
```



x86

Registers

- **ecx** — “Counter Register”
  - Often used as a counter in loops.
  - For syscalls,
    - it stores the second argument (pointer to the data to write).
- **edx** — “Data Register”
  - Stores the third argument to a syscall
    - For write, that’s the number of bytes to print.

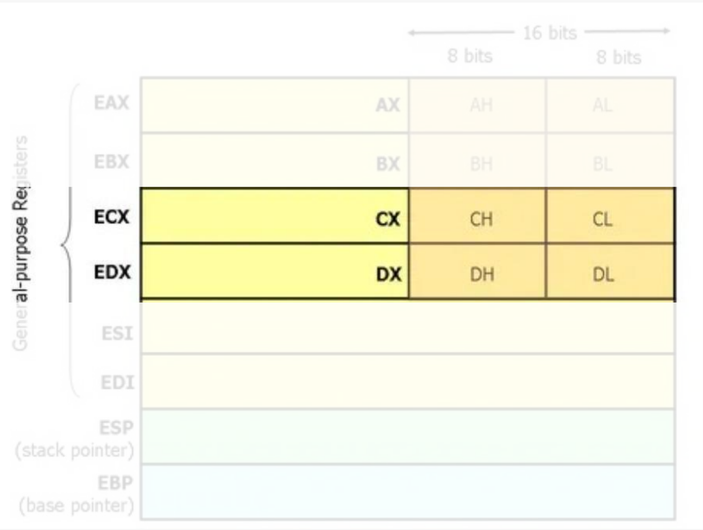
```
section .data
    result db 0, 10   ; space for one digit + newline

section .text
    global _start

_start:
    mov eax, 5        ; move 5 into eax register
    add eax, 3        ; eax = 8
    add al, '0'       ; convert to ASCII
    mov [result], al  ; store result

    mov eax, 4        ; sys_write
    mov ebx, 1        ; move 1 into ebx register
    mov ecx, result
    mov edx, 2
    int 0x80

    mov eax, 1
    xor ebx, ebx
    int 0x80
```



Registers

- All of these names:
  - eax,
  - ebx,
  - ecx,
  - edx

  are for 32-bit registers.

- Their 8-bit low parts are named
  - al, bl, cl, dl.

- Using the al in this code to convert to ASCII.

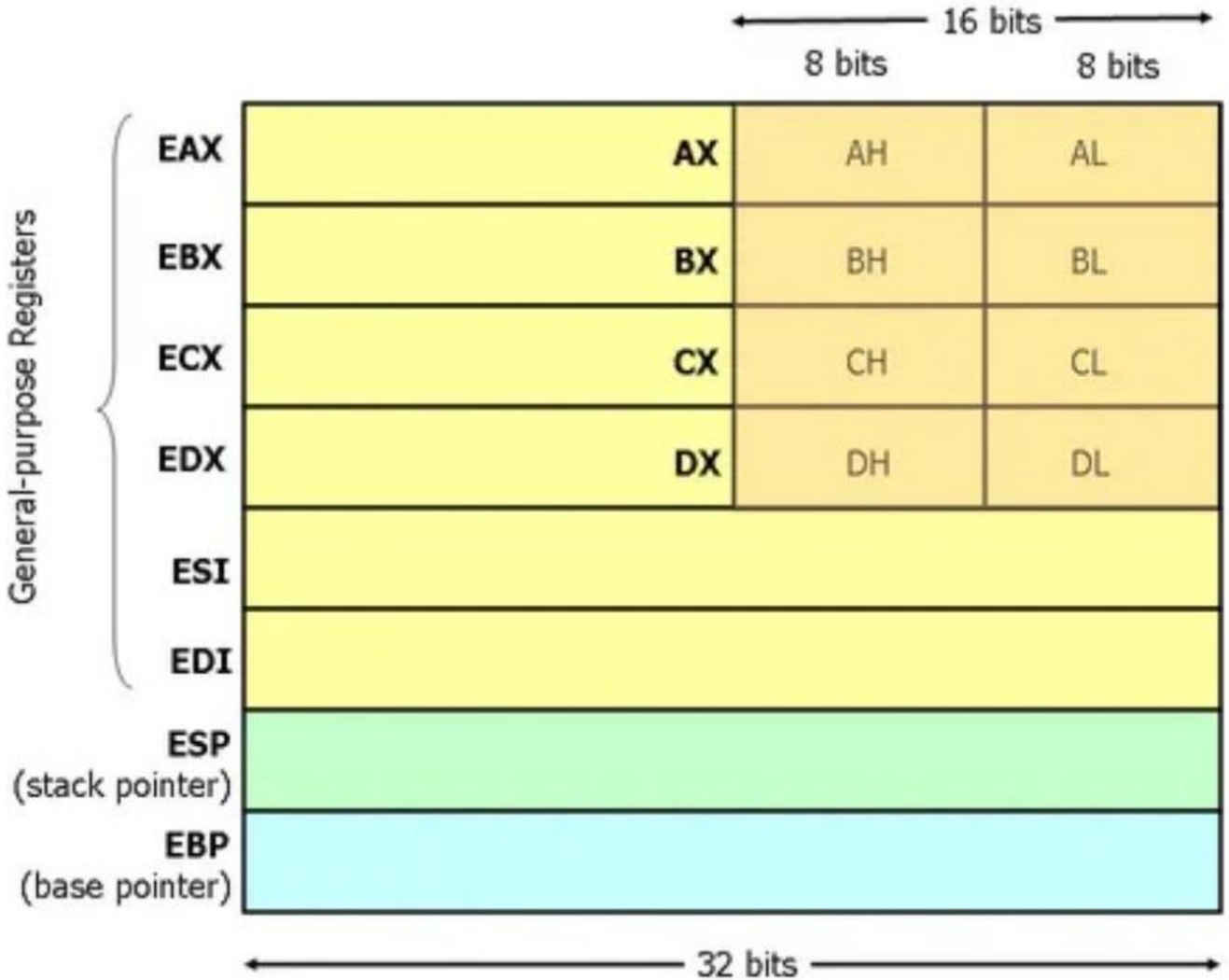
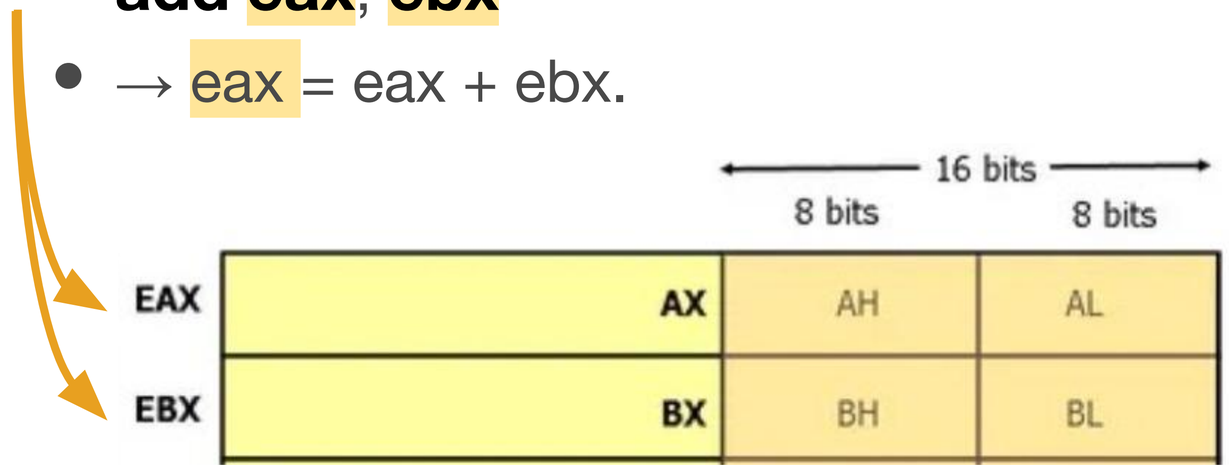


Figure 1. x86 Registers

x86

Instructions

- **add** — Adds values.
- Example:
  - mov eax, 5
  - **add eax**, **3**
  - → eax = 5 + 3.
  - **add eax**, **ebx**
  - → eax = eax + ebx.




```
section .data
    result db 0, 10   ; space for one digit + newline

section .text
    global _start

_start:
    mov eax, 5          ; move 5 into eax register
    add eax, 3          ; eax = 8
    add al, '0'         ; convert to ASCII
    mov [result], al    ; store result

    mov eax, 4          ; sys_write
    mov ebx, 1          ; move 1 into ebx register
    mov ecx, result
    mov edx, 2
    int 0x80

    mov eax, 1
    xor ebx, ebx        ; set ebx to 0.
    int 0x80
```

x86

Instructions

- **mov** — Moves data from one place to another
  - register ↔ register,
  - register ↔ memory,
  - etc.
- Example:
  - **mov eax**, **5**
  - → puts the number **5** into **eax**.

```
section .data
    result db 0, 10   ; space for one digit + newline

section .text
    global _start

_start:
    mov eax, 5          ; move 5 into eax register
    add eax, 3          ; eax = 8
    add al, '0'         ; convert to ASCII
    mov [result], al    ; store result

    mov eax, 4          ; sys_write
    mov ebx, 1          ; move 1 into ebx register
    mov ecx, result
    mov edx, 2
    int 0x80

    mov eax, 1
    xor ebx, ebx        ; set ebx to 0.
    int 0x80
```

x86

Instructions

- **xor** — Bitwise XOR.
  - A common trick:
    - **xor** ebx, ebx
    - sets ebx to zero without using an immediate **mov** 0.

```
section .data
    result db 0, 10   ; space for one digit + newline

section .text
    global _start

_start:
    mov eax, 5        ; move 5 into eax register
    add eax, 3        ; eax = 8
    add al, '0'       ; convert to ASCII
    mov [result], al  ; store result

    mov eax, 4        ; sys_write
    mov ebx, 1        ; move 1 into ebx register
    mov ecx, result
    mov edx, 2
    int 0x80

    mov eax, 1
    xor ebx, ebx      ; set ebx to 0.
    int 0x80
```

EXERCISE

Exercise 2.3: ALU - x86



1. Copy / Rewrite the code from the example screenshot (slides) and find the answers.
2. Write **Assembly code** for: *[Calculate; Print; Exit]*

- **Addition,** when **R2=21** and **R3=12** (add)
- **Subtraction,** when **R2=21** and **R3=12** (sub)
- **Multiplication,** when **R2=21** and **R3=10** (imul)
- **Division,** when **R2=21** and **R3=10** (idiv) (Integer Division)

3. Save your code into a text file for your later reference – e.g. **X86_Add.txt**

https://www.tutorialspoint.com/compilers/online-assembly-compiler.htm



Key Points: ISAs with Assembly (x86)

- **x86 is a CISC architecture with variable-length instructions**
  - Uses eax, ebx, ecx, edx as primary 32-bit registers
- **Key instructions: mov, add, xor for data manipulation**
- **Syscalls use int 0x80 with eax for call number**

EXERCISE

Exercise 2.4: ALU

Compare the commands of those 3 applications for Assembly programming to understand the ISAs, using the problems:

Task:

- Write a program that:
  - **loads** the **decimal value 100** into a register,
  - then **loads** the **value 0x64** (hex for 100) into another register.

ARM: MOV R0, #100 / MOV R1, #0x64
MIPS: li $t0, 100 / ori $t1, $0, 0x64 (or li $t1, 0x64 if the assembler supports it)
x86: mov eax, 100 / mov ebx, 64h

Learning Objective:

- Basic instruction syntax, literal values (immediates), and different numeric formats.

EXERCISE

Exercise 2.5: ALU

Compare the commands of those 3 applications for Assembly programming to understand the ISAs, using the problems:

Task:

- Write a program to **calculate** the **expression** c = a + b - 3.
- Assume the values for a (e.g., 15) and b (e.g., 10) are already in registers.

ARM: ADD R2, R0, R1 / SUB R2, R2, #3 (2-operand destination)
MIPS: add $t2, $t0, $t1 / subi $t2, $t2, 3 (3-operand destination)
x86: mov ecx, eax / add ecx, ebx / sub ecx, 3 (2-operand destination that overwrites)

Learning Objective:

- Fundamental arithmetic operations and the need for a separate register for the result



Key Points: Programming: ISAs with Assembly

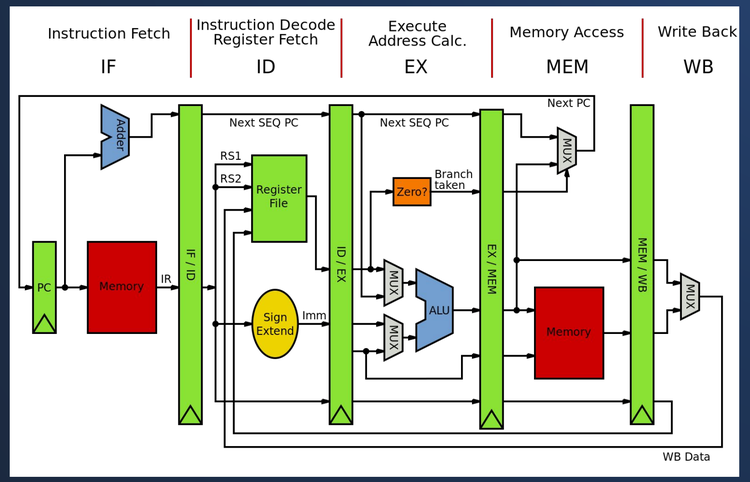
- **x86 Assembly**
  - CISC architecture (Intel 8086 family)
  - Registers: eax, ebx, ecx, edx
  - Instructions: mov, add, xor, int
- **Architecture Selection**
  - ARM (mobile),
  - MIPS (education/embedded systems)
  - x86 (laptop/ desktop/ server)

PART E

Types of Pipelining:

("Look ahead instructions" or staggered instruction sequences)

Arithmetic and instruction pipelining — two ways to overlap work.





	t1	t2	t3	t4	t5	t6	t7	t8	t9
ADD R3, R6, R5	IF	ID	IE	MEM	RW R3	--	--	--	--
SUB R4, R3, R5	--	IF	ID R3	IE	MEM	RW	--	--	--
OR R6, R3, R7	--	--	IF	ID R3	IE	MEM	RW	--	--
AND R8, R3, R9	--	--	--	IF	ID R3	IE	MEM	RW	--
XOR R10, R3, R11	--	--	--	--	IF	ID R3	IE	MEM	RW

Types of Pipelining

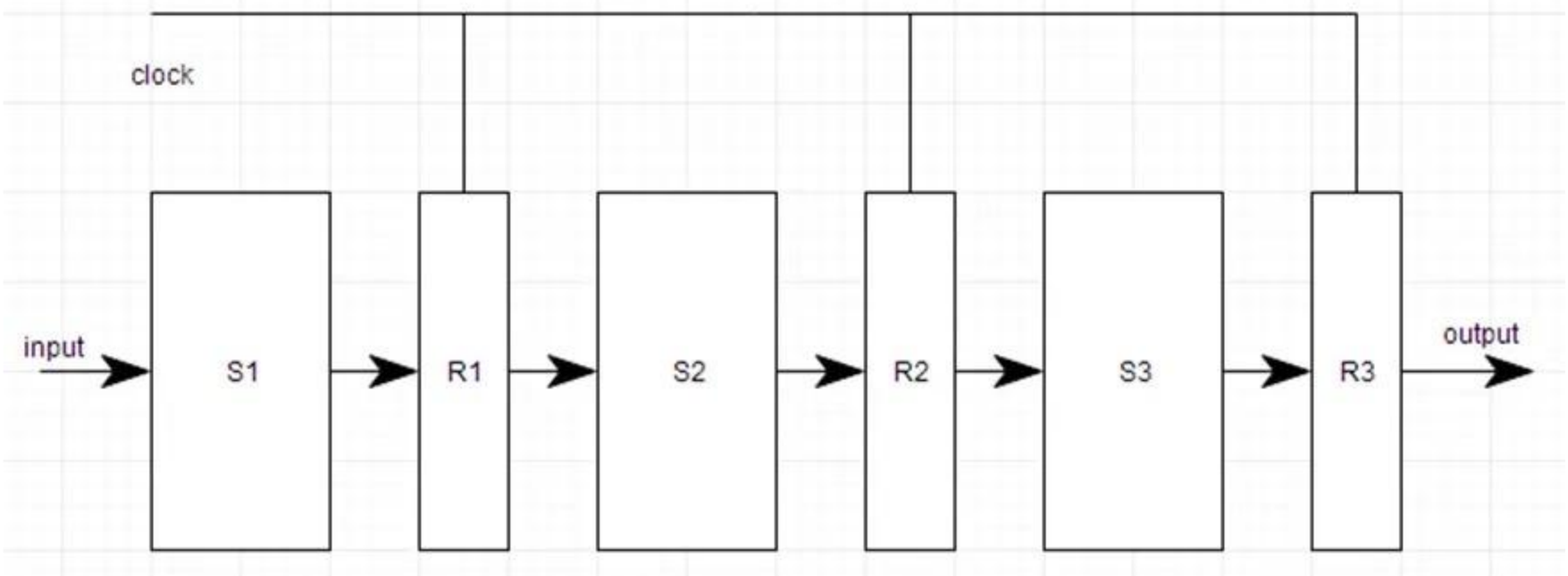
- In 1977, Handler and Ramamoorthy separated pipelining processors according to their performance.
  - **Arithmetic Pipelining**
  - **Instruction Pipelining**

Arithmetic Pipelining

- Arithmetic **Pipelining** is designed to calculate the addition, multiplication, and division of a floating-point number at high speed.
- Many arithmetic logic units are built into the system to perform parallel arithmetic calculations in a variety of data formats.
  - Examples of **arithmetic** pipelined processors:
    - Star-100
    - TI-ASC
    - Cray-1
    - Cyber-205

Instruction Pipelining

- A **number** of **instructions are pipelined** (*queued*), and the execution of the current instruction overlaps with the execution of the next instruction.
- It is also known as instruction lookahead.

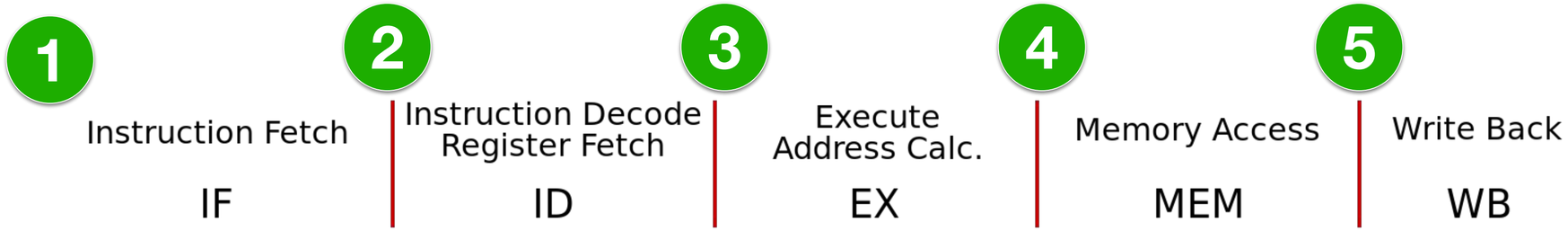




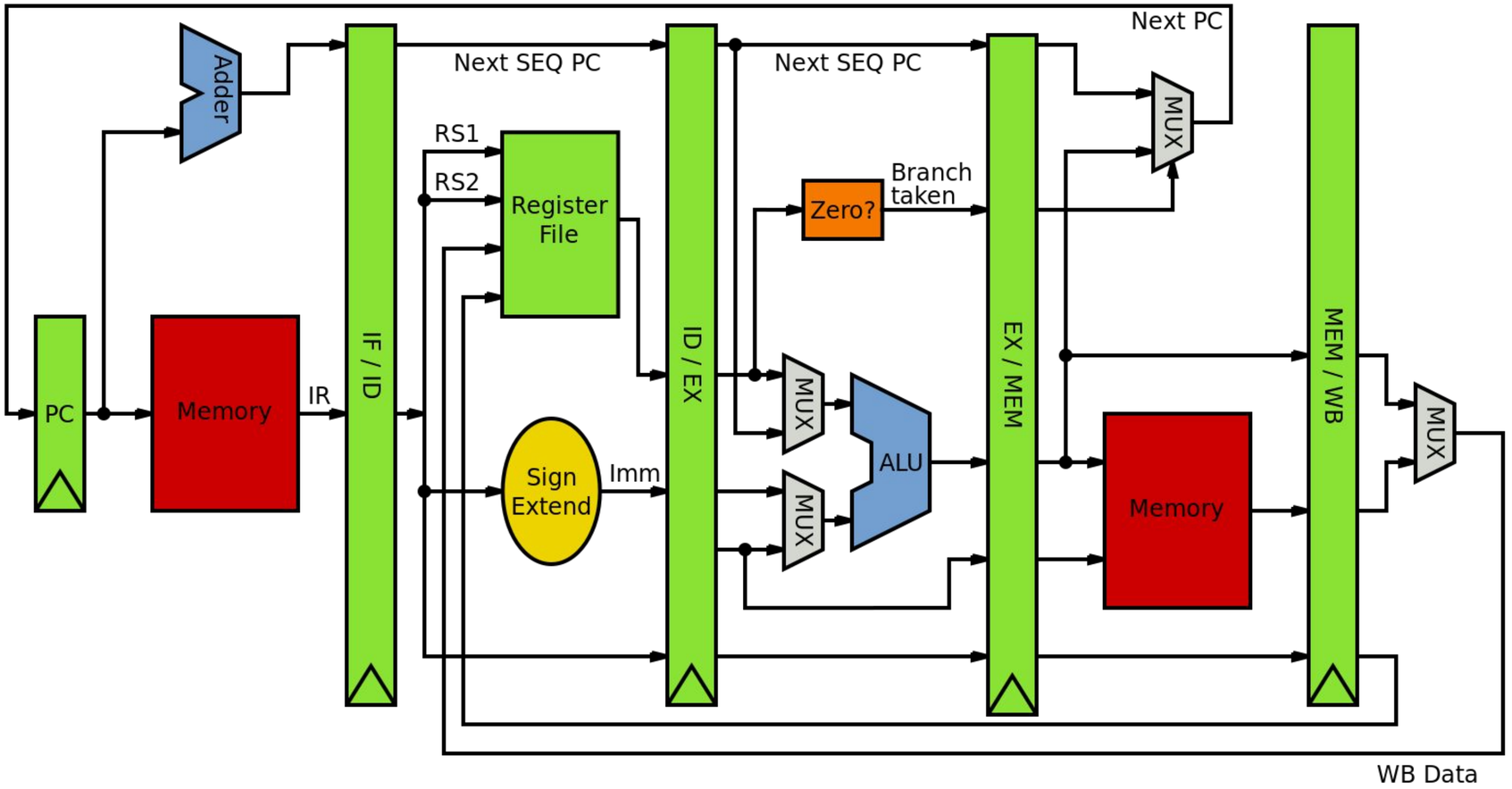
Key Points: Types of Pipelining

- **Arithmetic pipelining speeds up FP operations**
- **Instruction pipelining overlaps fetch/decode/execute stages**
- **Both types improve throughput without faster clock speed** *(duty cycle).*

Pipeline Stages of Instruction Set Architectures: MIPS



5-stage **Pipeline**



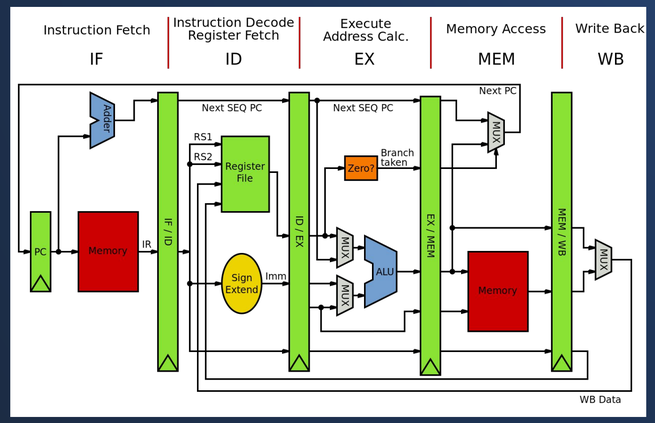
Stages:

1. Instruction Fetch
2. Instruction Decode & Register Fetch
3. Execute
4. Memory Access
5. Write Back

PART F

Pipelined Processors

How the five-stage pipeline executes many instructions at once.

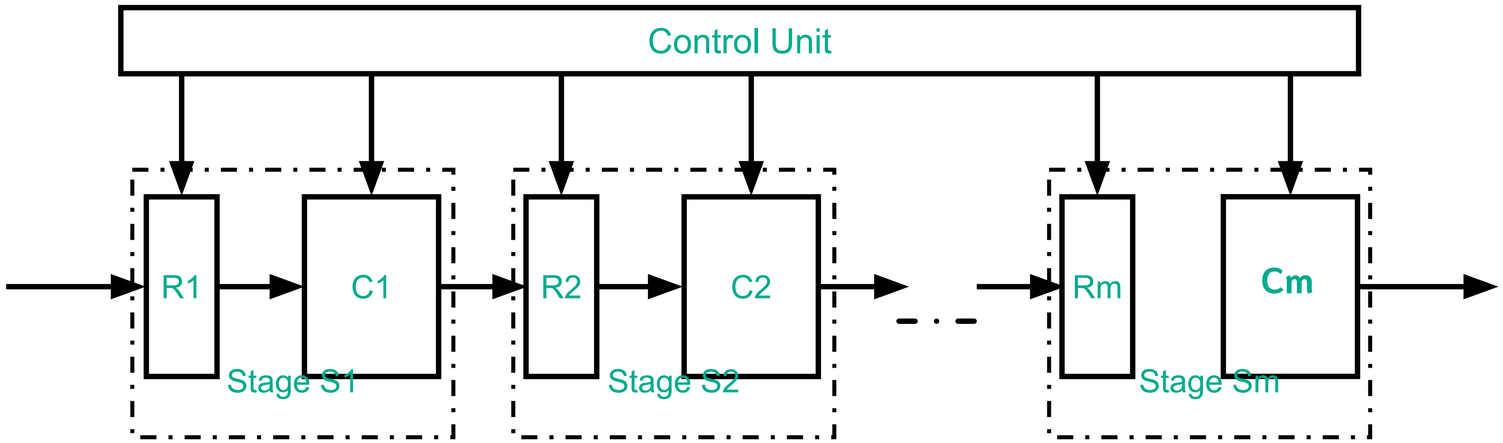




	t1	t2	t3	t4	t5	t6	t7	t8	t9
ADD R3, R6, R5	IF	ID	IE	MEM	RW R3	--	--	--	--
SUB R4, R3, R5	--	IF	ID R3	IE	MEM	RW	--	--	--
OR R6, R3, R7	--	--	IF	ID R3	IE	MEM	RW	--	--
AND R8, R3, R9	--	--	--	IF	ID R3	IE	MEM	RW	--
XOR R10, R3, R11	--	--	--	--	IF	ID R3	IE	MEM	RW

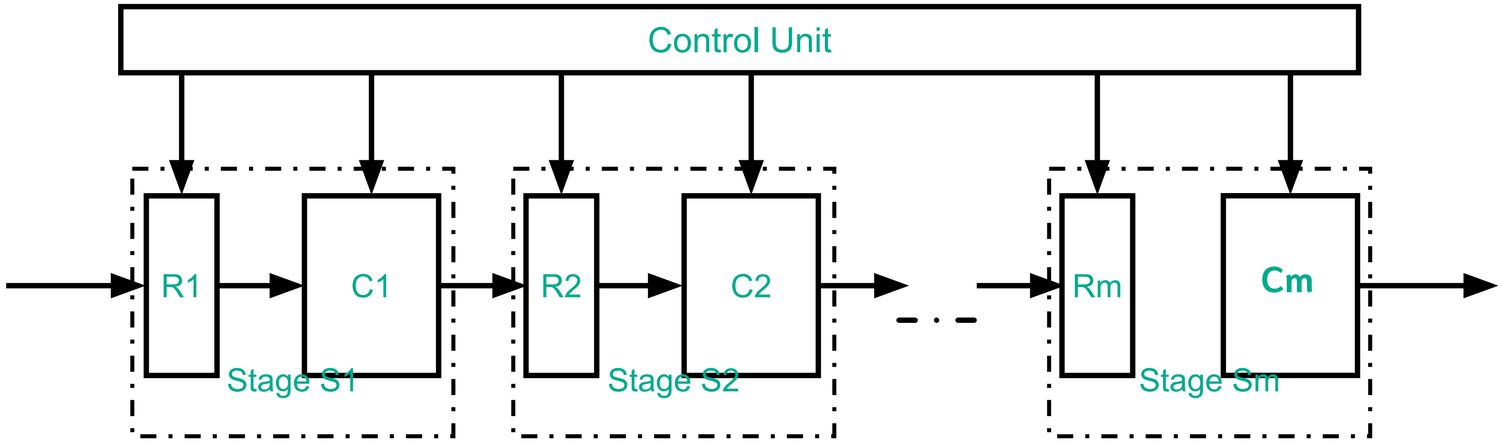
Pipelining

- **Pipelining** is the process of collecting instructions from a processor using a **pipeline**.
- **Pipelining** allows us to save and execute instructions through a systematic process.
- **Pipelining** is also called **pipeline** processing.



Pipelining

- Pipe installation is the method by which many instructions are concealed during execution.
- The pipe is divided into sections connected to form a pipe-like structure.
- Commands come in and out of another.
- Instructions come in from one end of the pipe and out of another end.
- It increases the total instruction throughput.



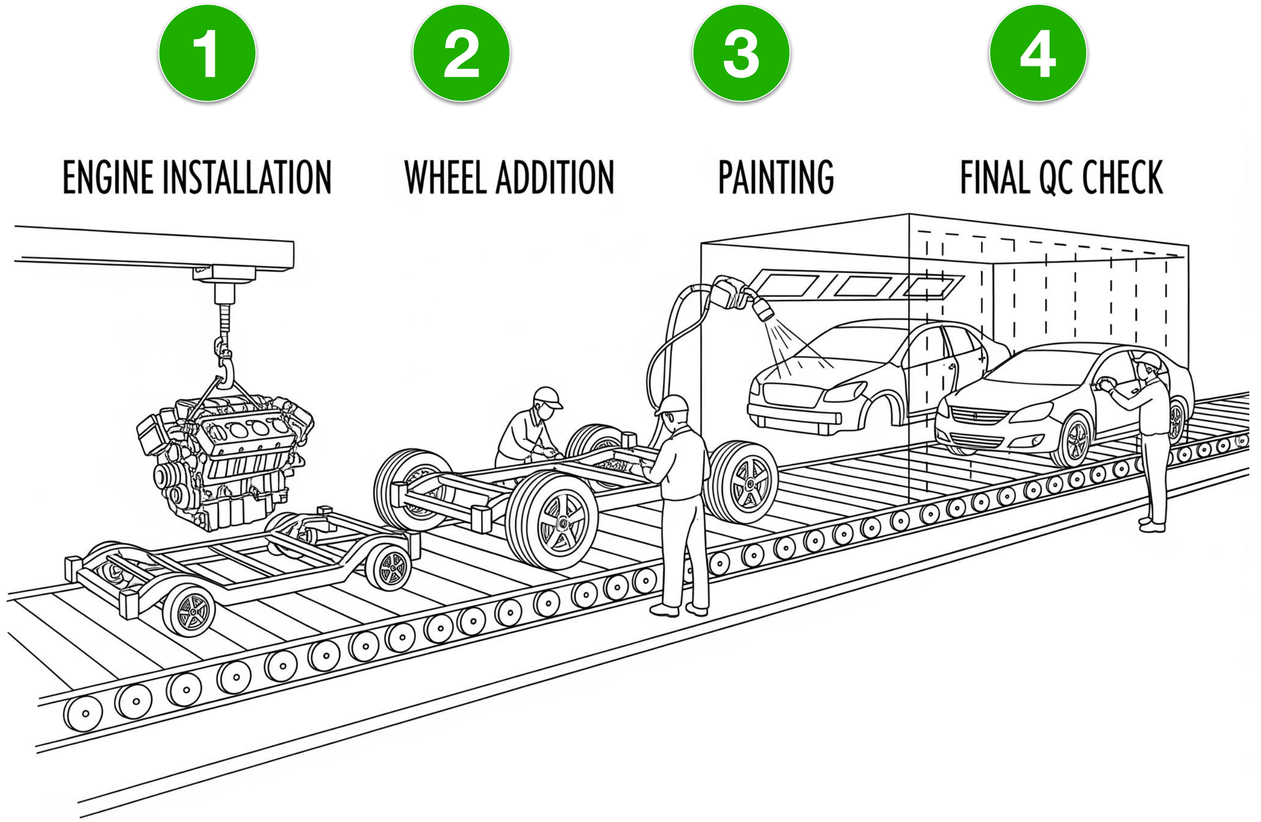
Pipelining

- While one instruction is being executed, others are moving through the earlier stages, allowing multiple instructions to be processed at the same time.

- This is similar to an assembly line in a factory,
  - where different workers (stages) handle different tasks on multiple products (instructions) simultaneously.

Pipeline Processor

- An **assembly line in a factory**, for example, consider how cars are built in a factory:
  - One worker installs the **engine**.
  - The next one adds the **wheels**.
  - Another **paints** the car.
  - The last one performs **final checks**.

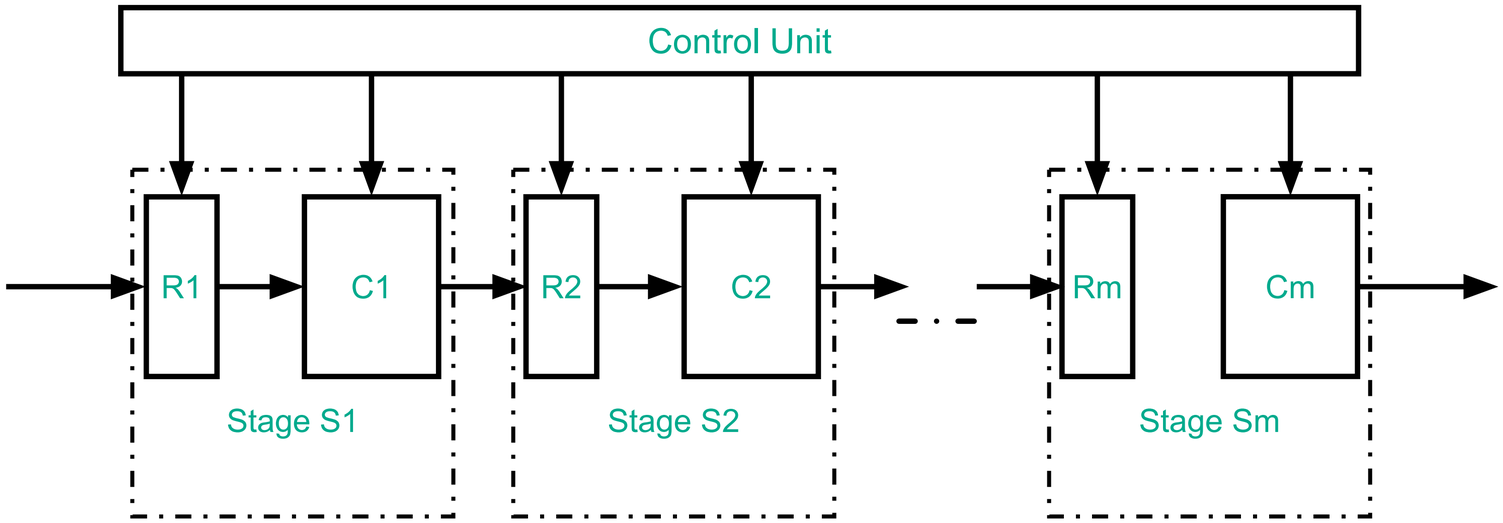


(While the next car chassis is waiting to start)

Pipeline Processor

- This overlap **increases** the overall **efficiency** and **throughput** of the processor.

- In **clock cycle 1**, data enters Stage S1 (R1 → C1).
- In **clock cycle 2**, the data moves to Stage S2 (R2 → C2), while new data enters Stage S1.



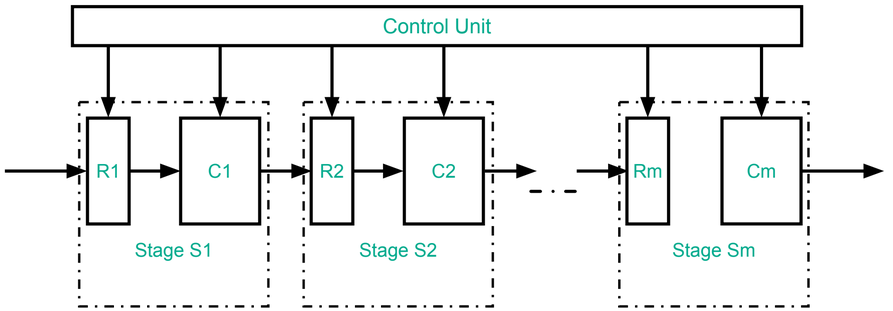
Pipeline Processor

- This process continues, and each stage is **simultaneously** **processing a different piece of data**.

- Eventually, the output appears at the end after passing through all stages.

Clock Cycle	Stage S1 (R1→C1)	Stage S2 (R2→C2)	Stage S3 (R3→C3)	...	Stage Sm (Rm→Cm)
1	Data A	-	-	...	...
2	Data B	Data A	-	...	...
3	Data C	Data B	Data A	...	...
4	Data D	Data C	Data B	...	...
5	Data E	Data D	Data C	...	...
...	...	...	...	...	...

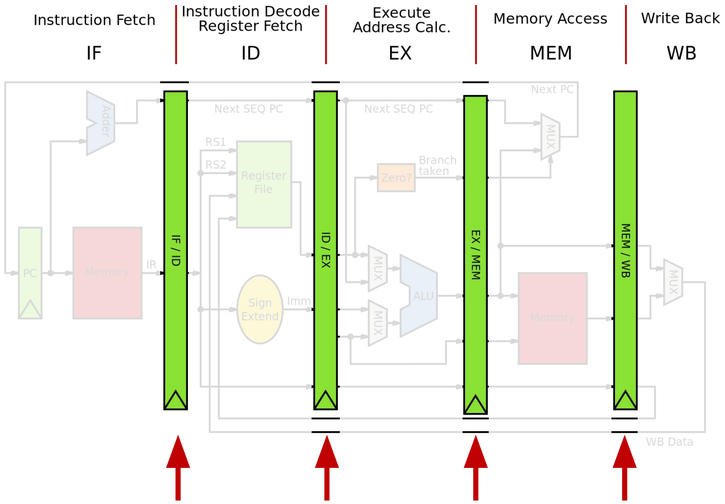
Pipelining



- In the **pipeline** system, each segment contains:
  - an **input register**
  - followed by a **combinational circuit**.

- The register holds data, and the combinational circuit performs functions on it.

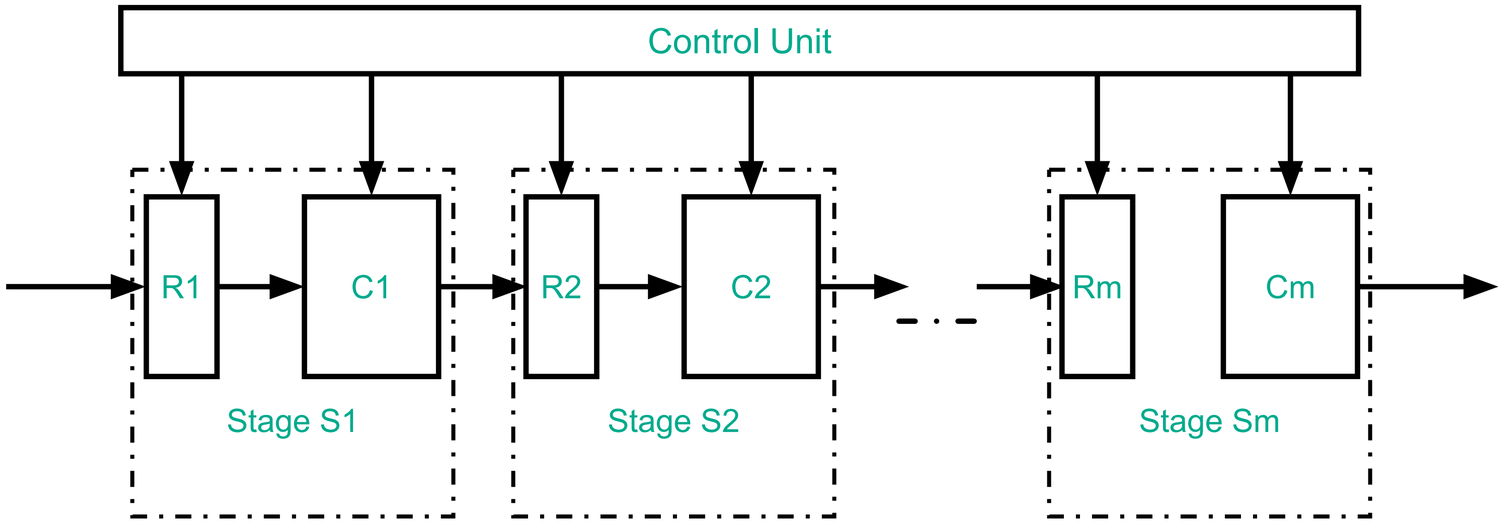
- The output of the combinational circuit is used in the input register of the next segment.



Pipeline Registers
(vertical green bars)

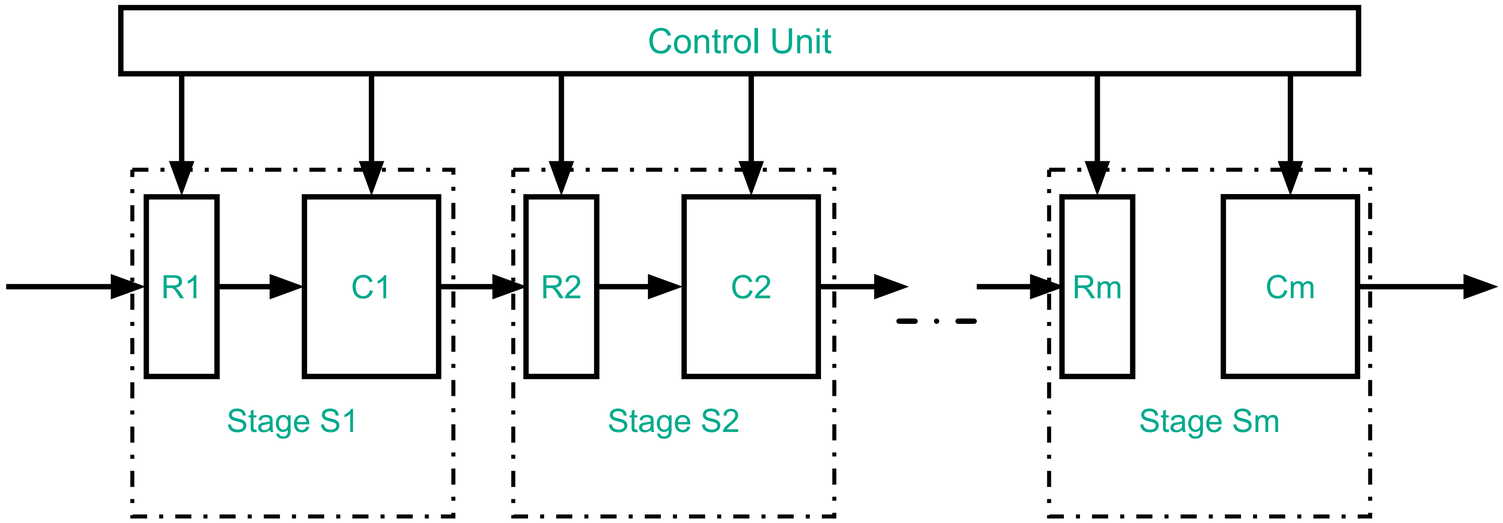
Pipelining: Components in the Diagram

- **Data In:**
  - This is the input data that enters the **pipeline**.
- **Stages (S1, S2, ..., Sm):** Each stage performs part of the operation.
  - The **pipeline** is divided into multiple stages (S1, S2, ..., Sm), and each stage handles a specific operation.
- **Data Out:** The final output after processing is complete across all **pipeline** stages.



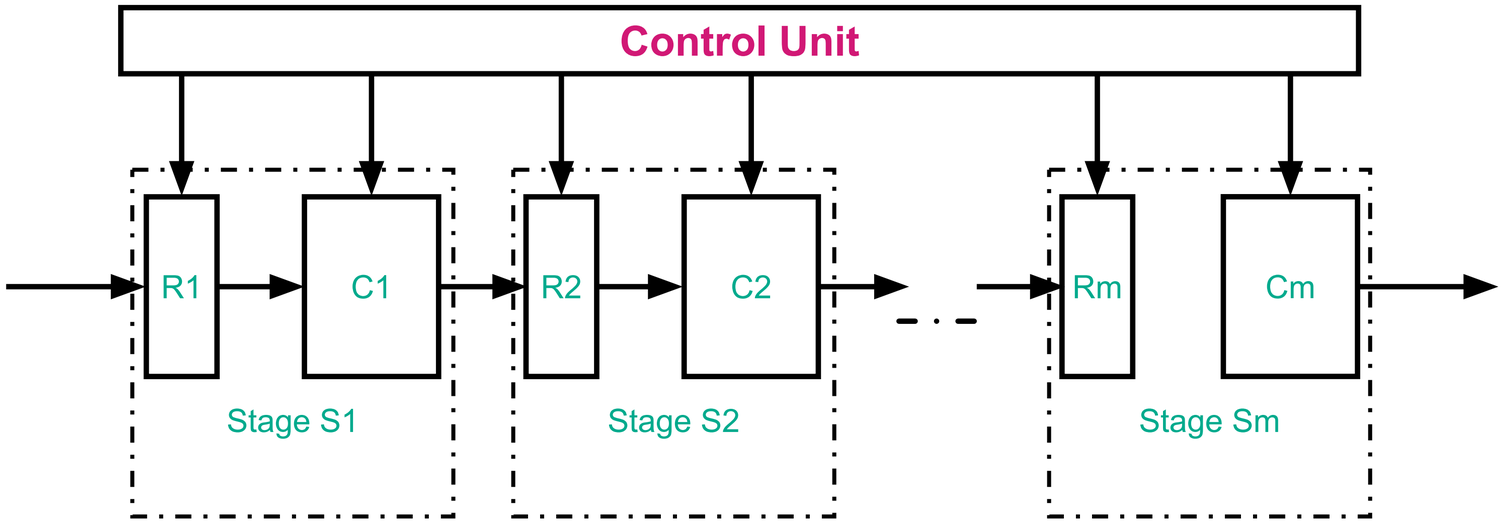
Pipelining: Components in the Diagram

- Registers **(R1, R2, ..., Rm):** These are **pipeline** registers.
  - They temporarily hold data between stages to ensure smooth transfer and isolation between operations.
  - Example: R1 stores the output of Stage S1 before passing it to S2.



Pipelining: Components in the Diagram

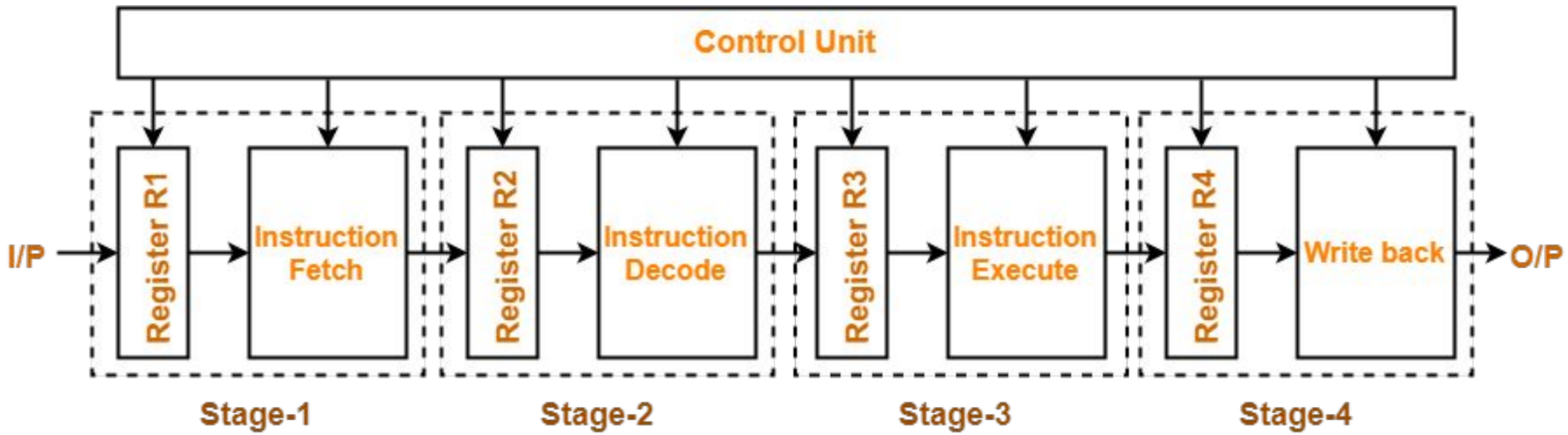
- **Computation Units (C1, C2, ..., Cm):**
  - These perform the actual processing (like arithmetic or logical operations by **ALU**).
    - *Each computation unit corresponds to a specific stage. (e.g. IF, ID, EX, MEM, WB)*
- **Control Unit:** Manages the timing and control signals for each stage.
  - Ensures that each stage **operates in sync** and processes the correct data at the right time.



Pipeline Processor

- A **pipeline processor** is a type of CPU architecture that improves processing speed by dividing instruction execution into separate stages.

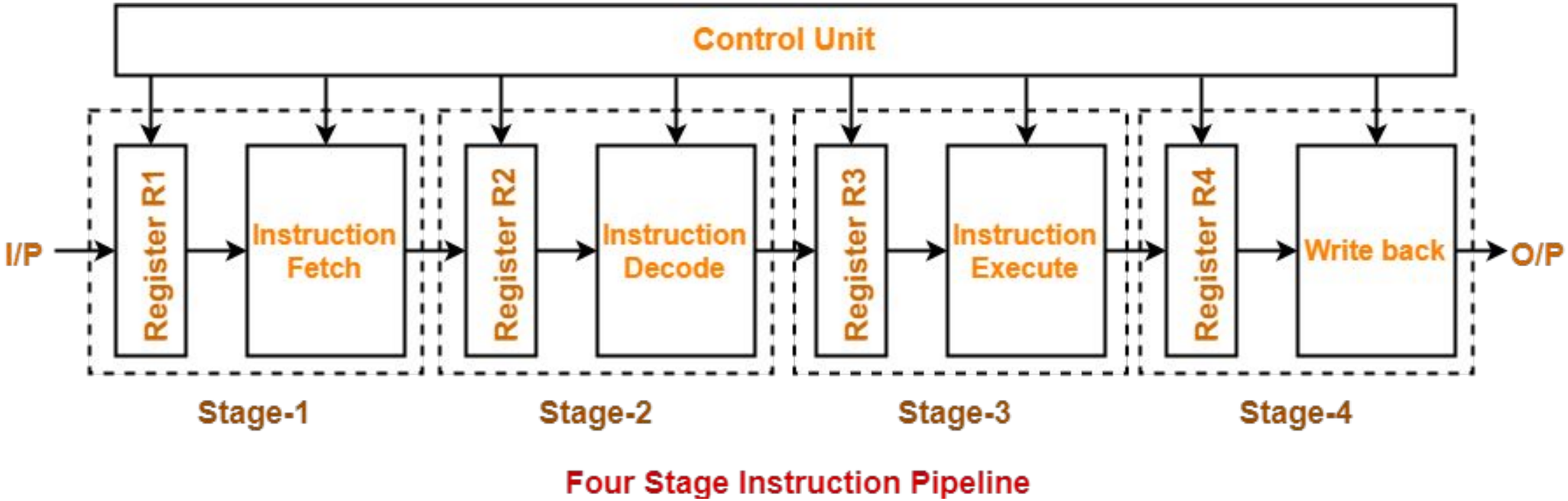
- Each stage handles a specific part of the instruction, such as *fetching*, *decoding*, *executing*, **memory access**, and *writing* results.



Four Stage Instruction Pipeline

source: https://www.gatevidyalay.com/pipelining-in-computer-architecture/

Non-Pipelined vs. Pipelined Executions

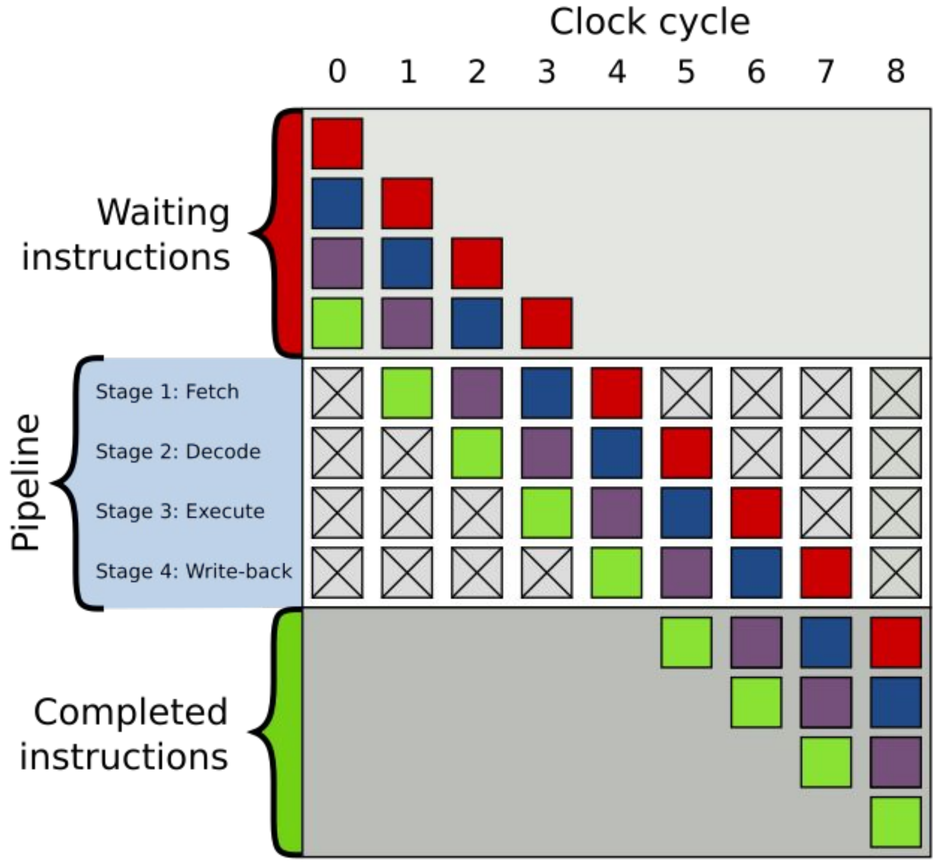


Four Stage Instruction Pipeline

source: https://www.gatevidyalay.com/pipelining-in-computer-architecture/

Basic 4-stage Pipeline

- The following is an execution of the same four instructions through a 4-stage **pipeline**.



source: https://en.wikipedia.org/wiki/Pipeline_stall

Basic 5-stage Pipeline

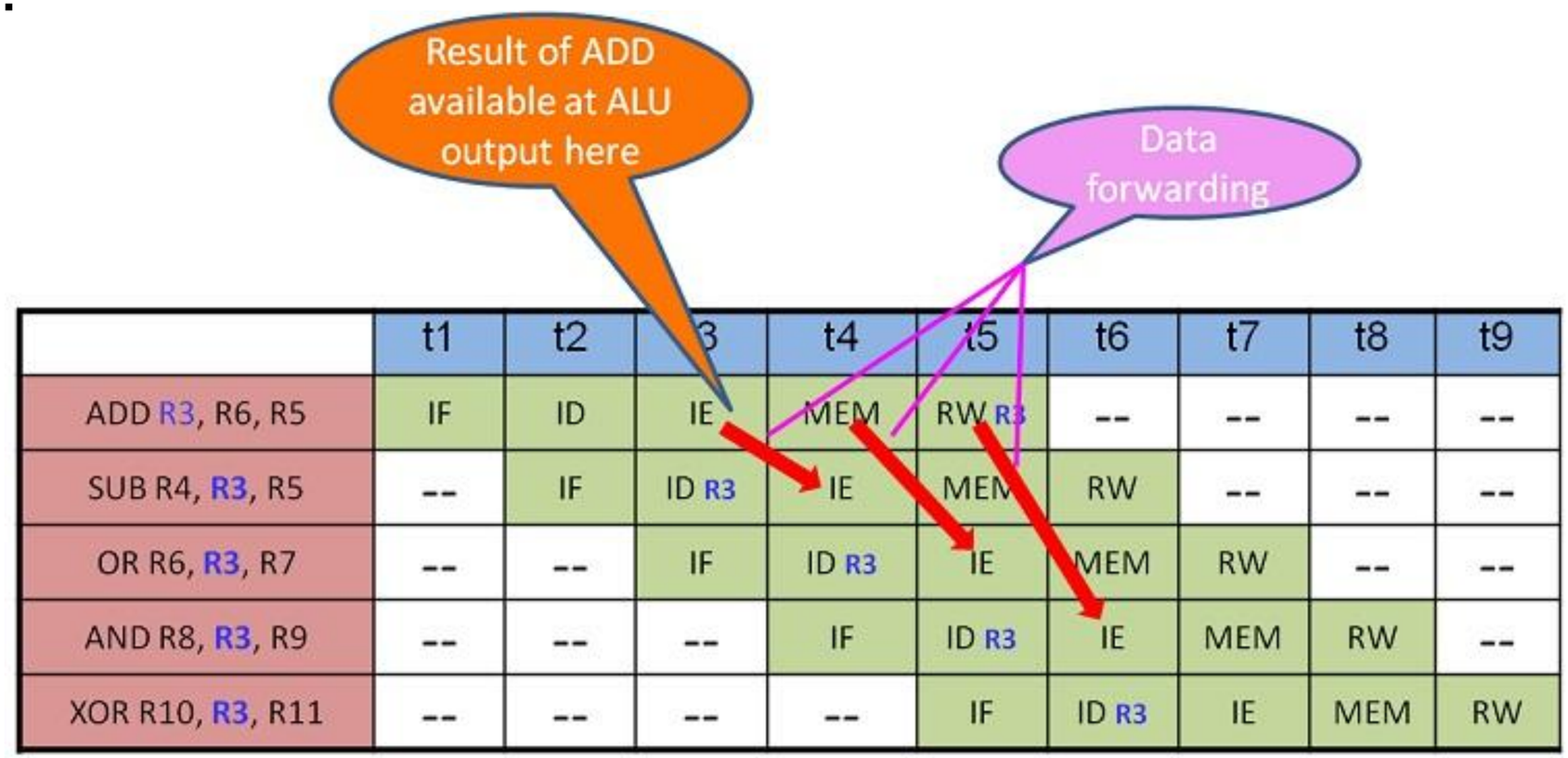
- The following is an execution of the same four instructions through a 5-stage **pipeline**.

Clock cycle / Instr. No.	1	2	3	4	5	6	7
1	IF	ID	EX	MEM	WB		
2		IF	ID	EX	MEM	WB	
3			IF	ID	EX	MEM	WB
4				IF	ID	EX	MEM
5					IF	ID	EX

source: https://en.wikipedia.org/wiki/Instruction_pipelining

Basic 5-stage Pipeline

- The following is an execution of the same four instructions through a 5-stage **pipeline**.



	t1	t2	t3	t4	t5	t6	t7	t8	t9
ADD R3, R6, R5	IF	ID	IE	MEM	RW R3	--	--	--	--
SUB R4, R3, R5	--	IF	ID R3	IE	MEM	RW	--	--	--
OR R6, R3, R7	--	--	IF	ID R3	IE	MEM	RW	--	--
AND R8, R3, R9	--	--	--	IF	ID R3	IE	MEM	RW	--
XOR R10, R3, R11	--	--	--	--	IF	ID R3	IE	MEM	RW

source: https://en.wikipedia.org/wiki/Instruction_pipelining

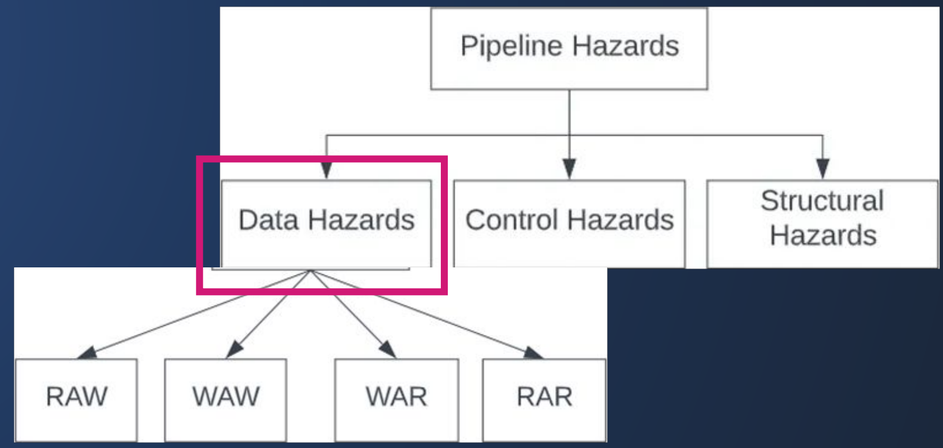


Key Points: Pipelined Processors

- **Pipeline divides instruction execution into stages**
  - IF, ID, EX, MEM, WB in classic 5-stage RISC pipeline
- **Pipeline registers store intermediate data between stages**
  - IF/ID, ID/EX, EX/MEM, MEM/WB register banks
- **Throughput improves but individual instruction latency unchanged**

PART G.1

Pipeline Hazards



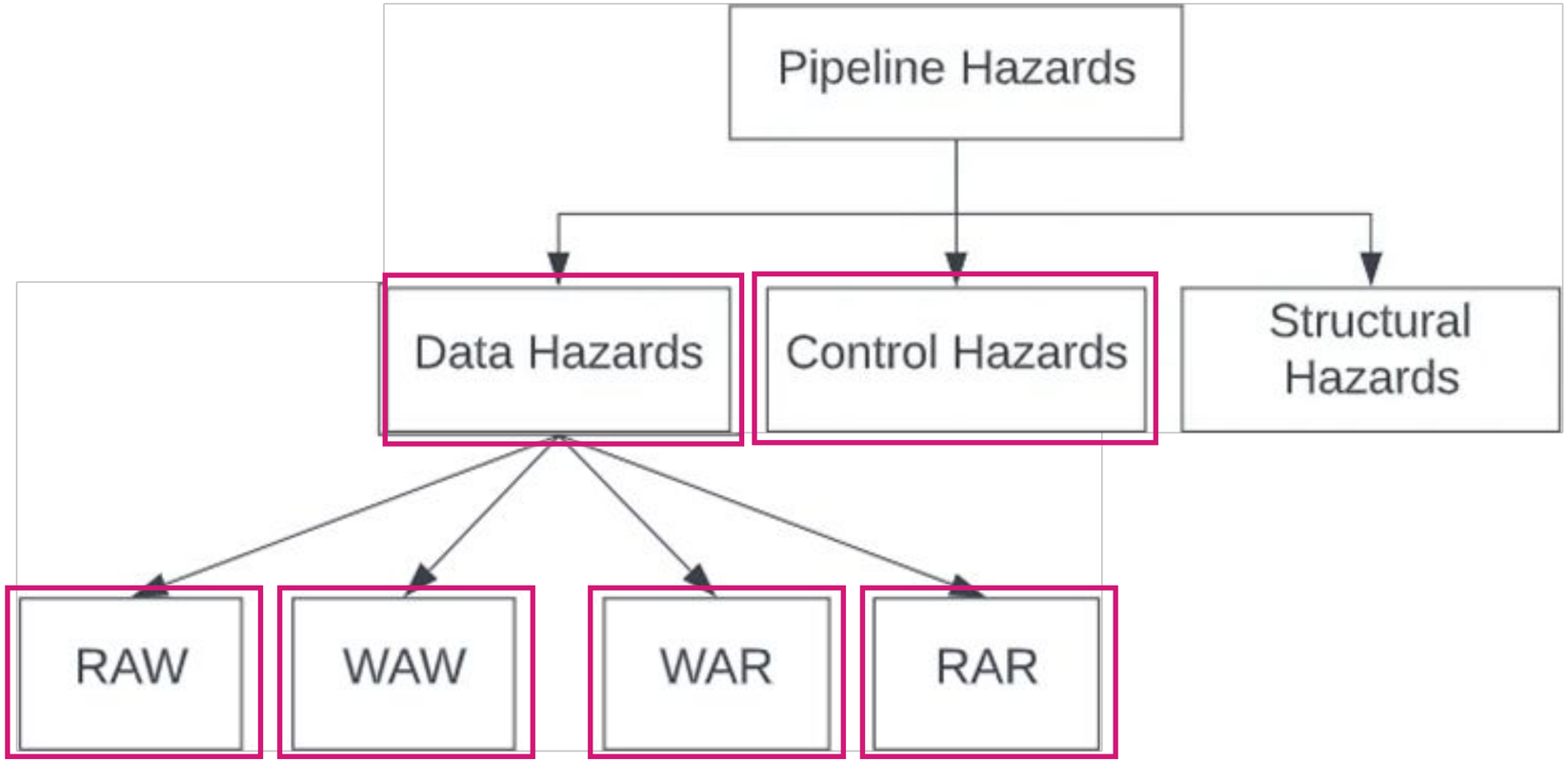
When pipelining goes wrong: data and control hazards, stalls and forwarding.

Pipeline Hazards

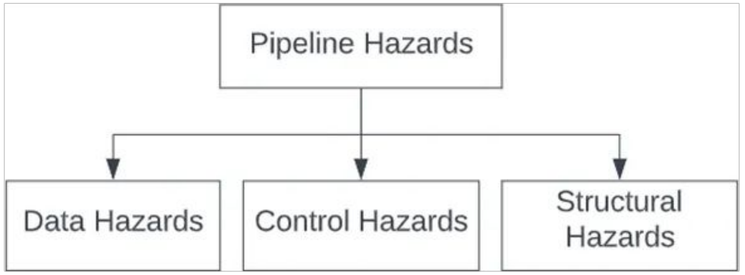
- A **pipeline** must stop for some reason, which is called pipeline hazards.
  - Hazards are problems that occur in the **pipeline**.
  - Rising hazards in the **pipeline** prevent the subsequent instruction from executing during its fixed clock cycle.
  - Hazards reduce the performance from the ideal speedup gained by **pipelining**.

Pipeline Hazards

- There are mainly three types of hazards:



Data Hazards: Overview

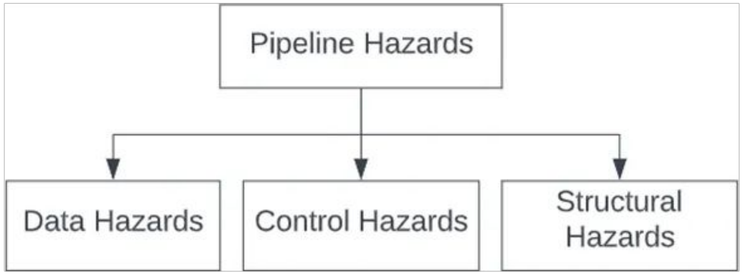


- The **ALU** instruction data hazard is caused by:
  - an ALU instruction writes a new value to a register (e.g., R1), and the register cannot be updated immediately.
- However, as two separate instructions can use the same storage; each must be executed in the correct sequence.

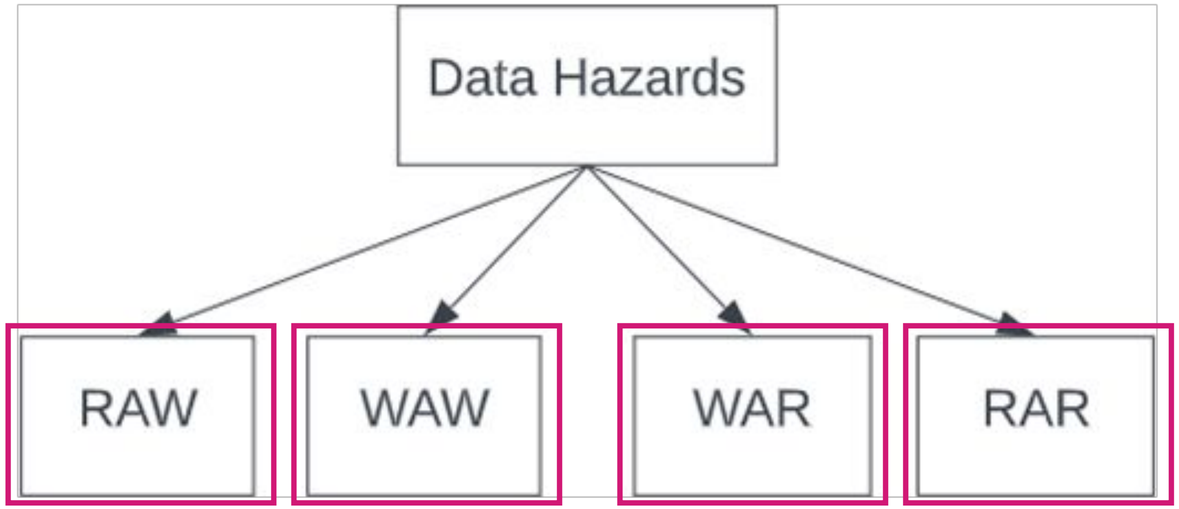
an instruction is dependent on the outcome of a preceding instruction $\wedge$ if the previous instruction's result has not yet been computed $\Rightarrow$ data hazards will occur

source: https://medium.com/@shrutika.gade22/pipeline-hazards-f6c317824b9f

Data Hazards

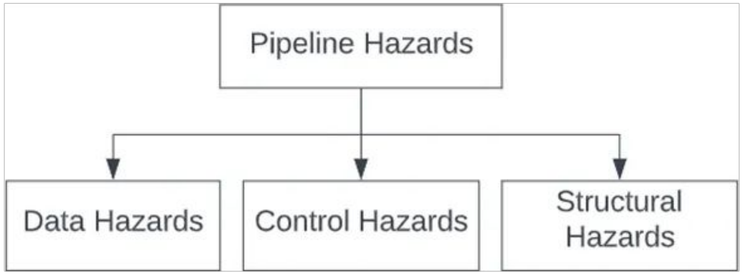


- There are four kinds of Data Hazards:



source: https://medium.com/@shrutika.gade22/pipeline-hazards-f6c317824b9f

Write after Write (WAW)



- **Write after Write (WAW)**:

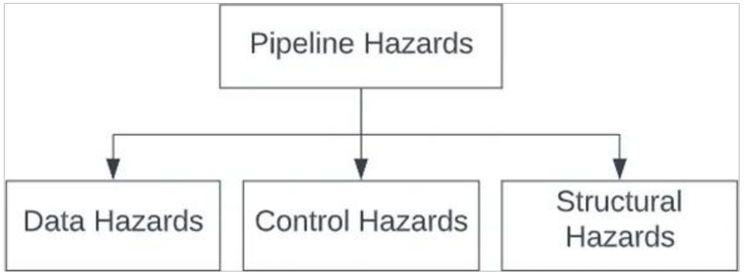
  Output dependency is sometimes known as WAW.

- These hazards arise when an instruction's output register is used for writing after it has been written by a previous instruction.

- For example,

```
ADD R1, — , — ;
SUB R1, — , — ;
```

Write after Read (WAR)



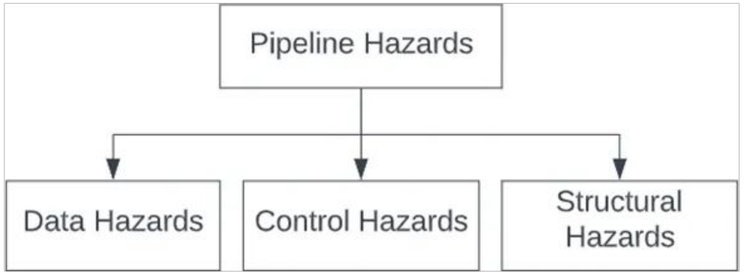
- **Write after Read (WAR)**: Anti-dependency is often known as WAR.

- These hazards arise when an instruction's output register is used after it has been read by a previous instruction.

- For example,

```
ADD — , R1, — ;
SUB R1, — , — ;
```

Read after Read (RAR)



- **Read after Read (RAR)**:

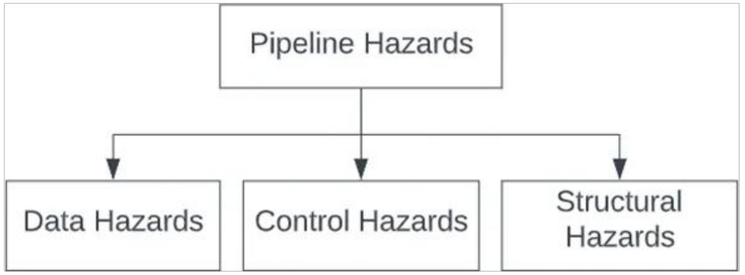
  When two instructions read from the same register, RAR occurs.

- For example,

  ADD — , R1, — ;
  SUB — , R1, — ;

- These Read after Read (RAR) hazards **do not present an issue for the processor** because reading a register value does not modify the register value.

Read after Write (RAW)



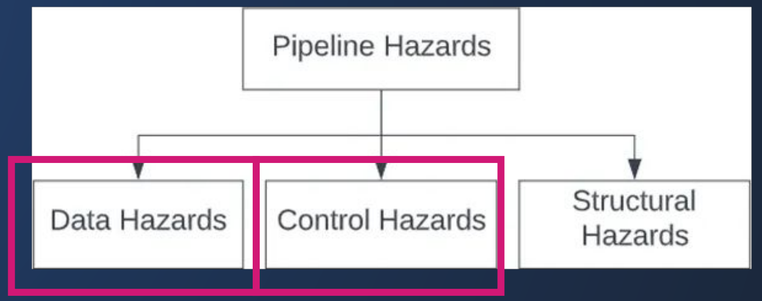
- **Read after Write (RAW)**: True dependency or Flow dependency are other terms for RAW.
- It happens when a value generated by one instruction is required by another instruction.
- For example,
  - These hazards require the **use of stalls**.

```
ADD R1, — , — ;
SUB — , R1, — ;
```

  - A **pipeline** stall is a delay in the execution of an instruction to resolve a hazard.)

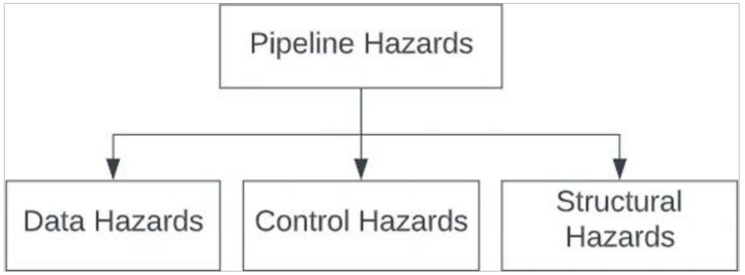
PART G.2

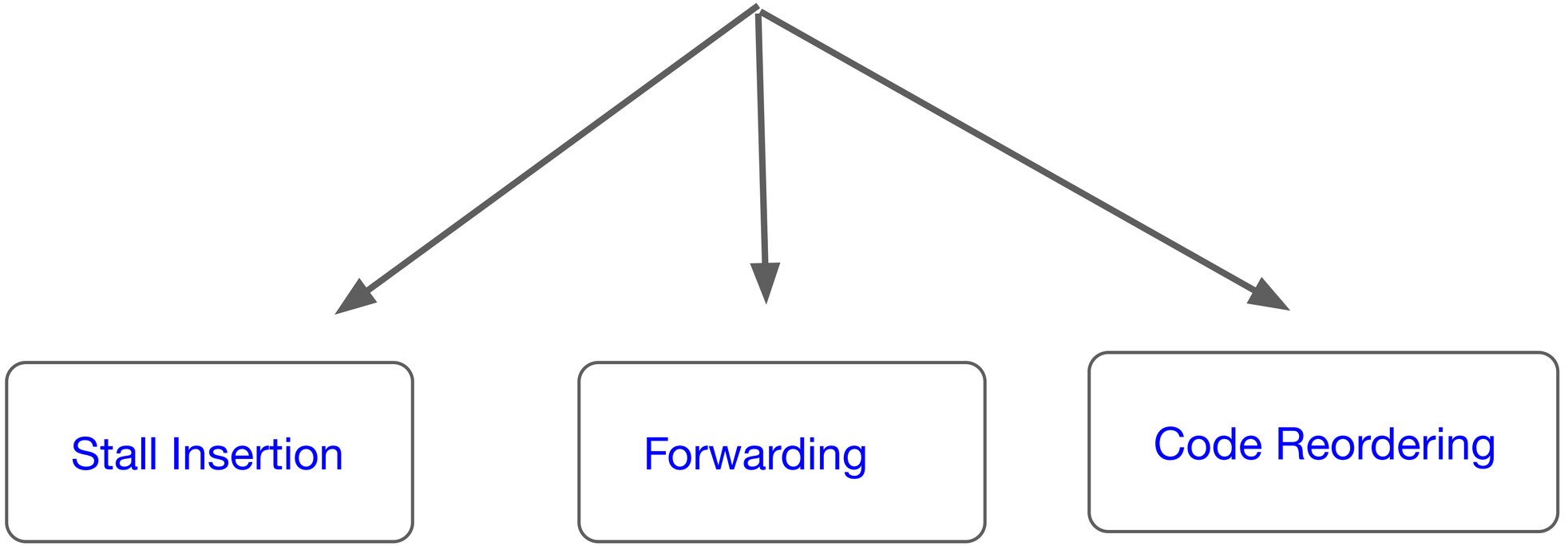
Pipeline Hazards: Handling



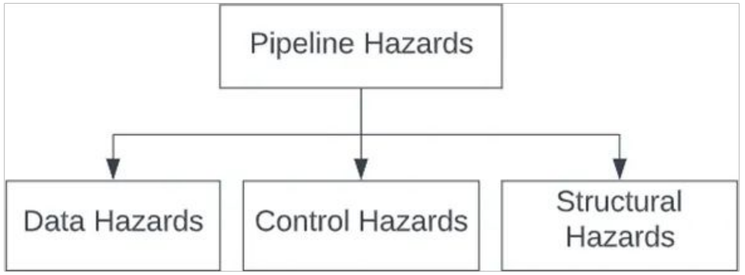
When pipelining goes wrong: data and control hazards, stalls and forwarding.

Data Hazards: Handling





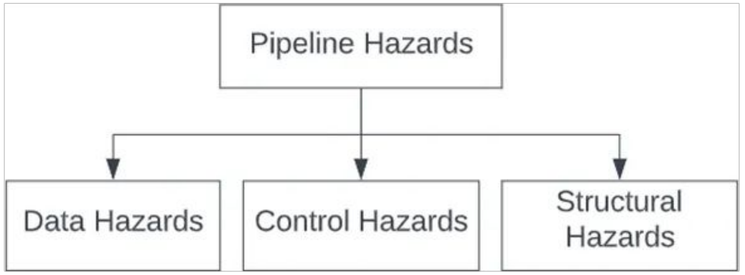
Data Hazards: Handling



Stall Insertion

1. Stall Insertion:
   - This method inserts one or more stalls (no-op instructions: “bubble”) into the **pipeline**.
   - **Delays execution** of the current instruction until the required operand is written to the **register file**.
   - This method **reduces pipeline efficiency** and **throughput**.

Data Hazards: Handling



Forwarding

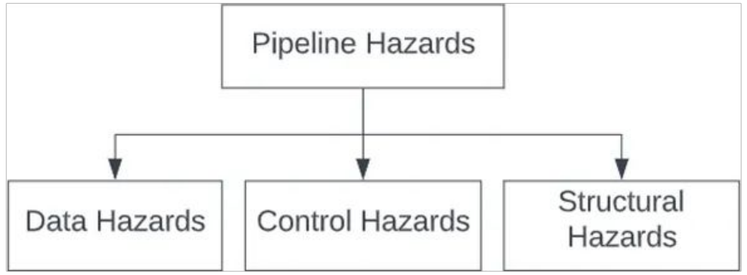
Forwarding:

It modifies the **pipeline** by adding special circuitry.

- This method works because the required values take less time to go via a circuit (wire) than a **pipeline segment** does to compute its result.

2.

Data Hazards: Handling

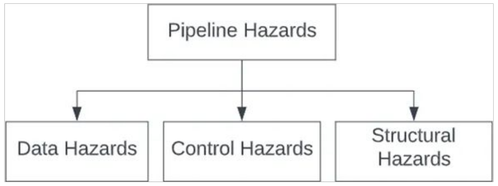


Forwarding

Code Reordering:

- A **hardware-dependent compiler** (specialized software) is required to reorder code instructions.

Stall Insertion



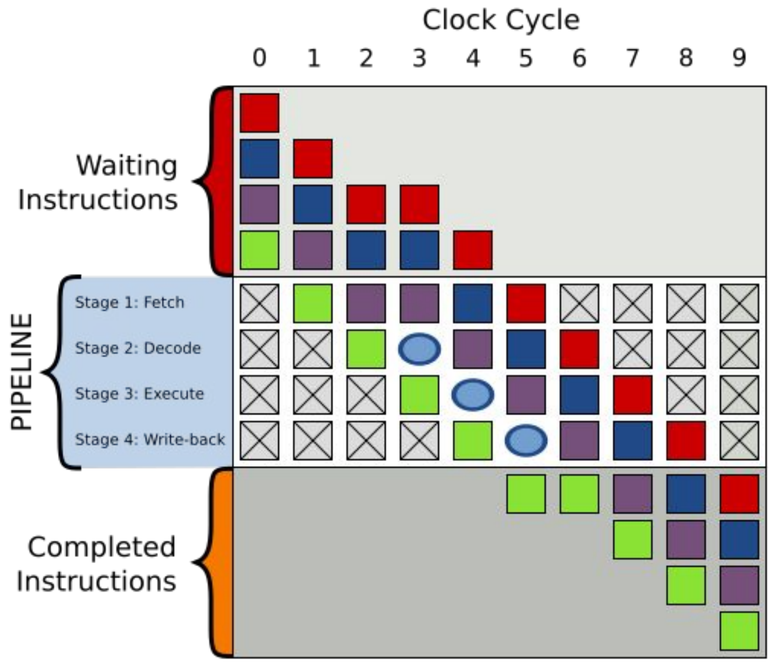
- **Read after Write (RAW)**:

  True dependency or Flow dependency are other terms for RAW.

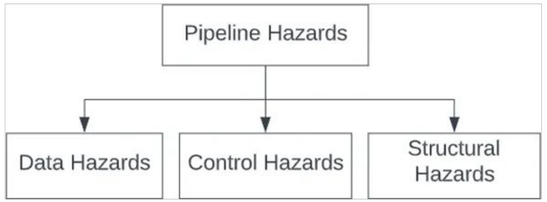
- It happens when a value generated by one instruction is required by another instruction.

  ADD R1, — , — ;

  SUB — , R1, — ;



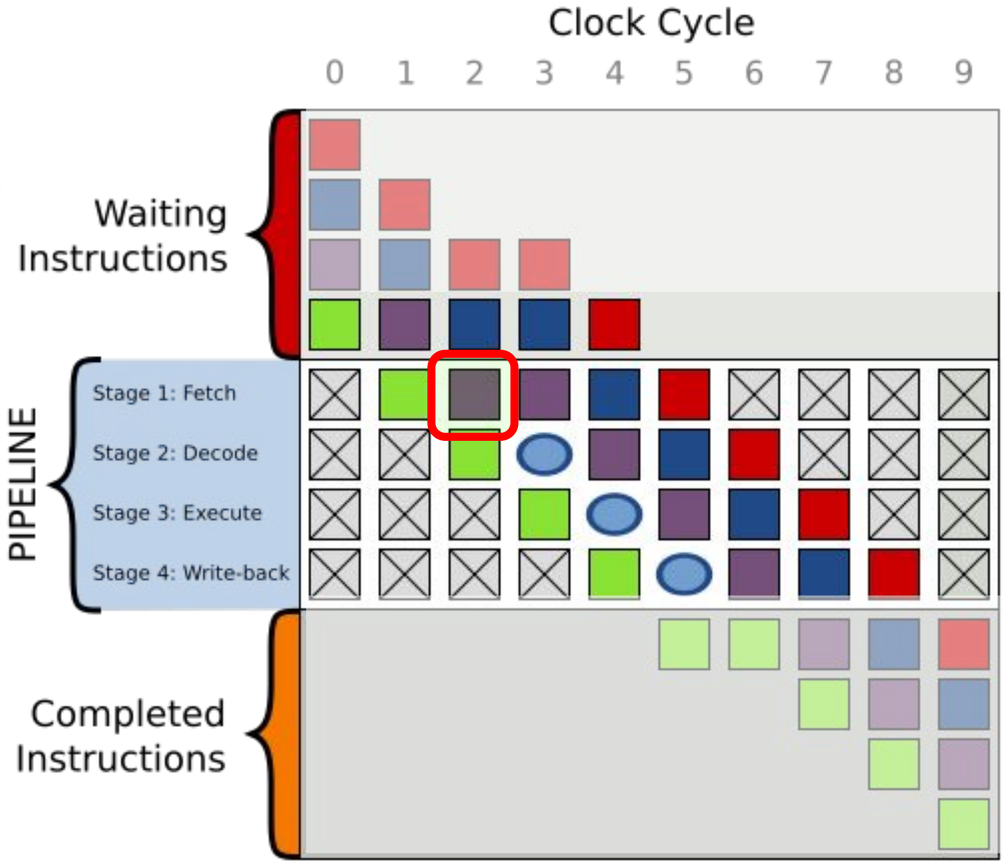
Stall Insertion: Example



- A **delay in fetching** (stall) the **purple instruction in cycle #2** leads to a **bubble** being created.

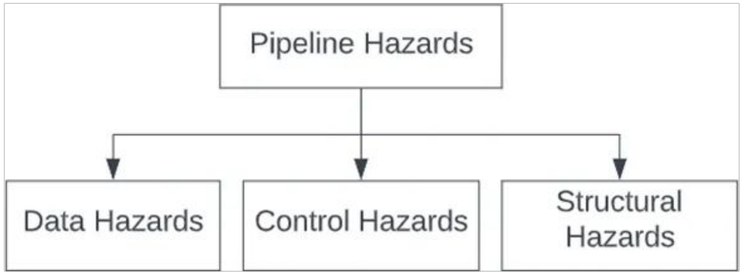


- All instructions after it are delayed as well.

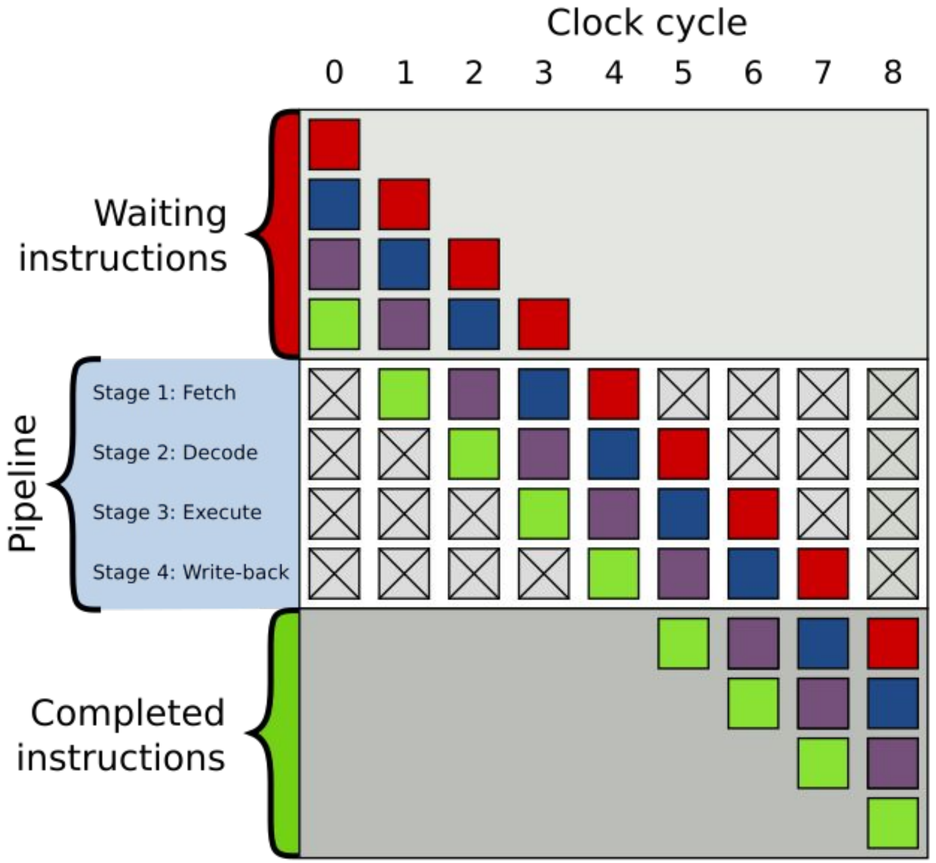


source: https://en.wikipedia.org/wiki/Pipeline_stall

Stall Insertion



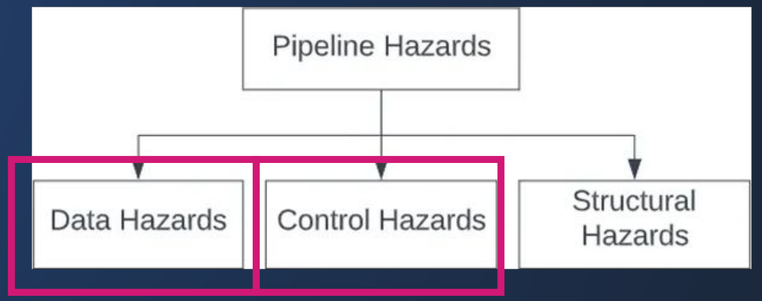
- No stall; same example.
- Without a delay in fetching the purple instruction in cycle #2.
- *No efficiency reduction*



source: https://en.wikipedia.org/wiki/Pipeline_stall

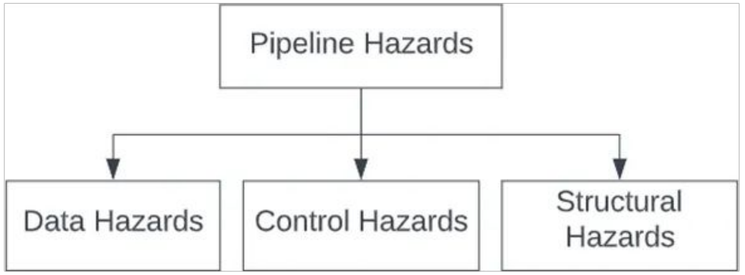
PART G.2

Pipeline Hazards: Handling
More advanced cases

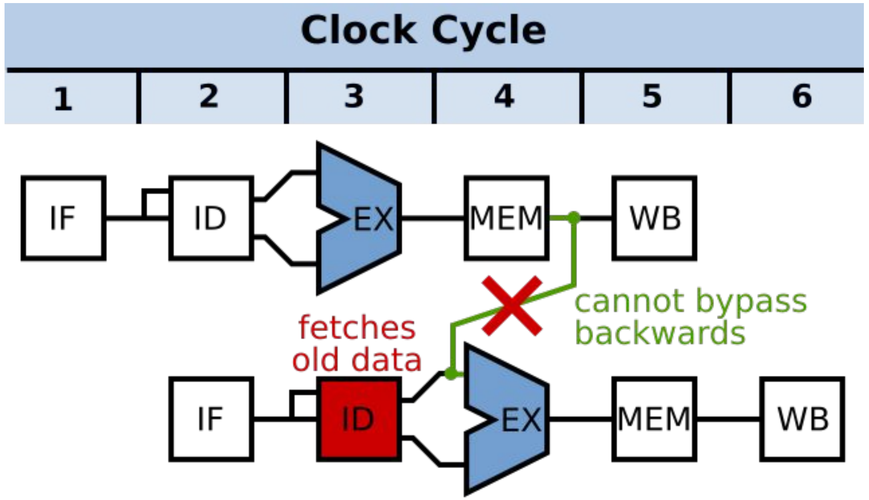


When pipelining goes wrong: data and control hazards, stalls and forwarding.

Classic RISC Pipeline



- A classic RISC **pipeline**, with five stages:
  - IF = **Instruction Fetch**
  - ID = **Instruction Decode**
  - EX = Execute
  - MEM = **Memory access**
  - WB = Register **write back**



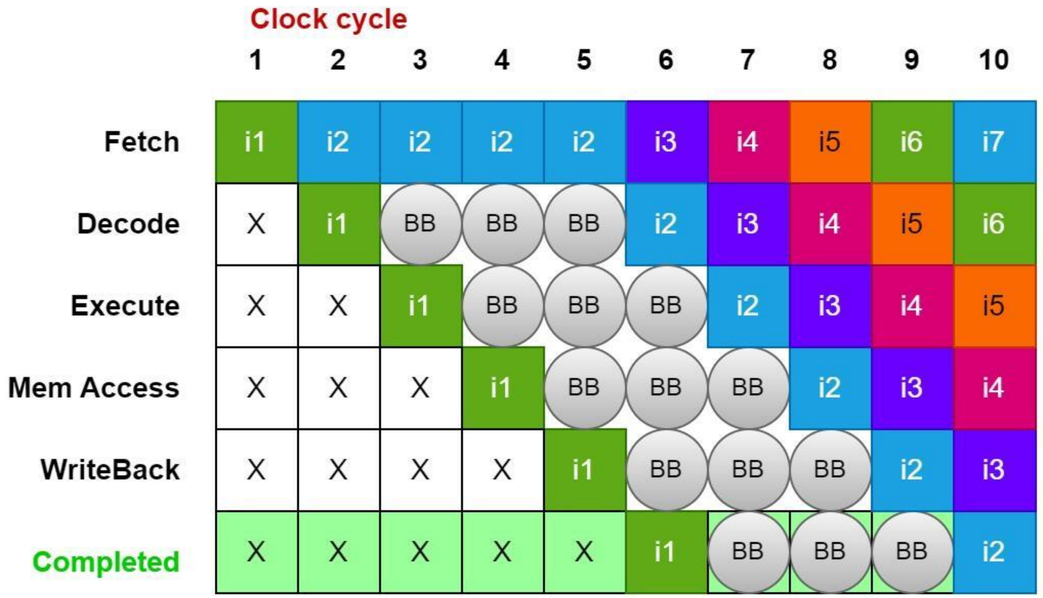
source: https://en.wikipedia.org/wiki/Pipeline_stall

- In this example,
  - data available after the MEM stage (4th stage) of the first instruction is **required** as input by the EX stage (3rd stage) of the second instruction.

Stall Insertion

- The stalls occur in the 5-stage **pipeline**; the pipeline may be forced to stall until the instruction can be successfully executed.

- We ideally expect our CPU to work at full throughput, i.e., CPI = 1.
- But when the **pipeline** stalls, it reduces the throughput/performance of the CPU because CPI increases.



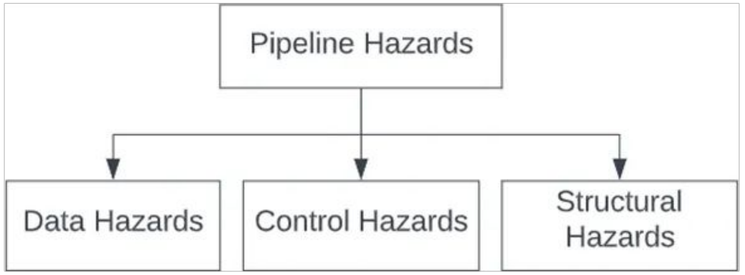
- For a non-ideal CPU:

$$\text{Clock cycles Per Instruction} = 1 + \text{Stall cycles per instruction.}$$

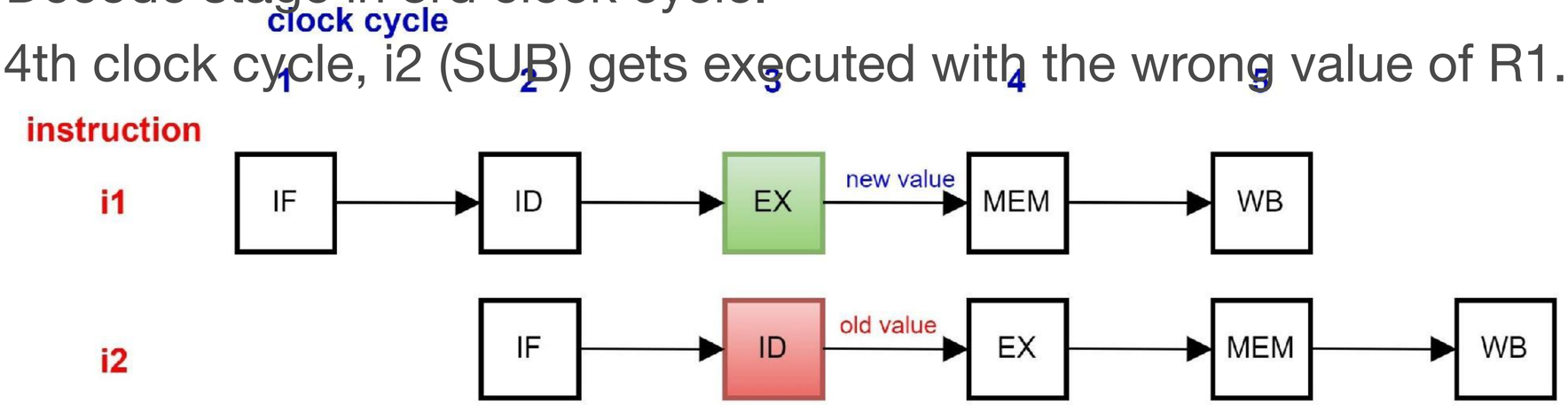
$$\implies \text{CPI}_{\text{non-ideal}} = \text{CPI}_{\text{ideal}} + \text{Stall cycles per instruction}$$

source: https://chipmunklogic.com/digital-logic-design/designing-pequeno-risc-v-cpu-from-scratch-part-3-dealing-with-pipeline-hazards/

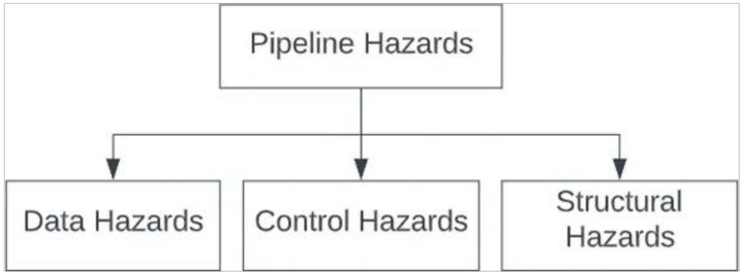
Data/Operand Forwarding



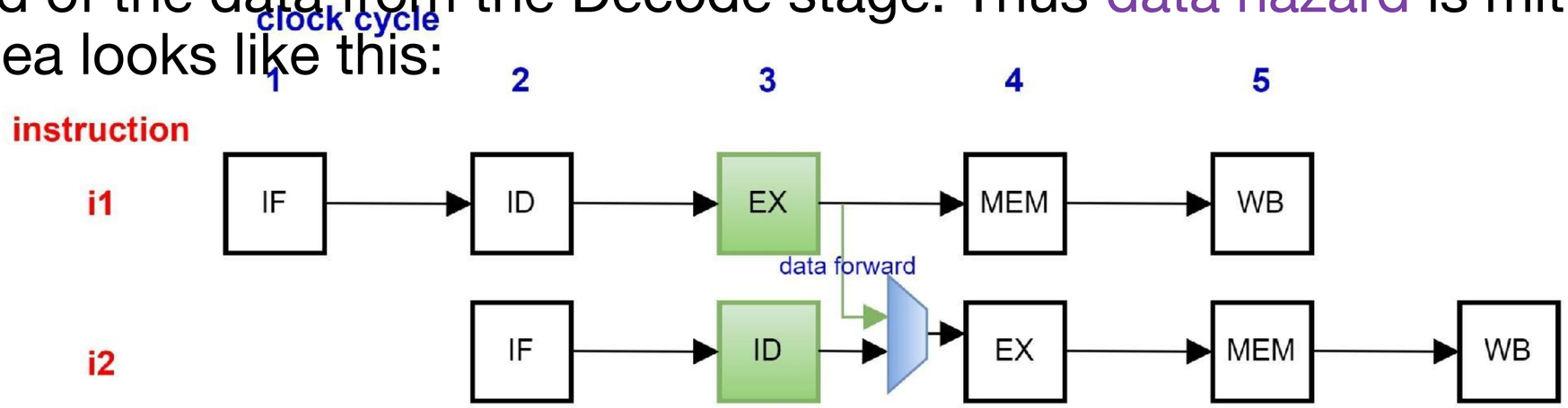
- Assume two adjacent instructions in a 5-stage **pipeline**, i1 (ADD) and i2 (SUB), have a RAW data dependency between them as both are accessing a register R1.
- The CPU should stall the instruction i2 (SUB) until i1 (ADD) writes back the result to the register R1.
- If the CPU has no stalling mechanism, the older value is read by i2 (SUB) from R1 at the Decode stage in 3rd clock cycle.
- In the 4th clock cycle, i2 (SUB) gets executed with the wrong value of R1.



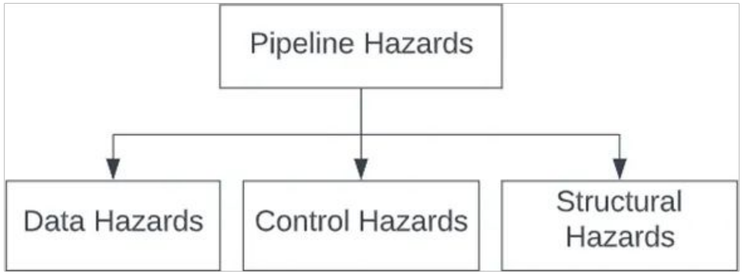
Data/Operand Forwarding



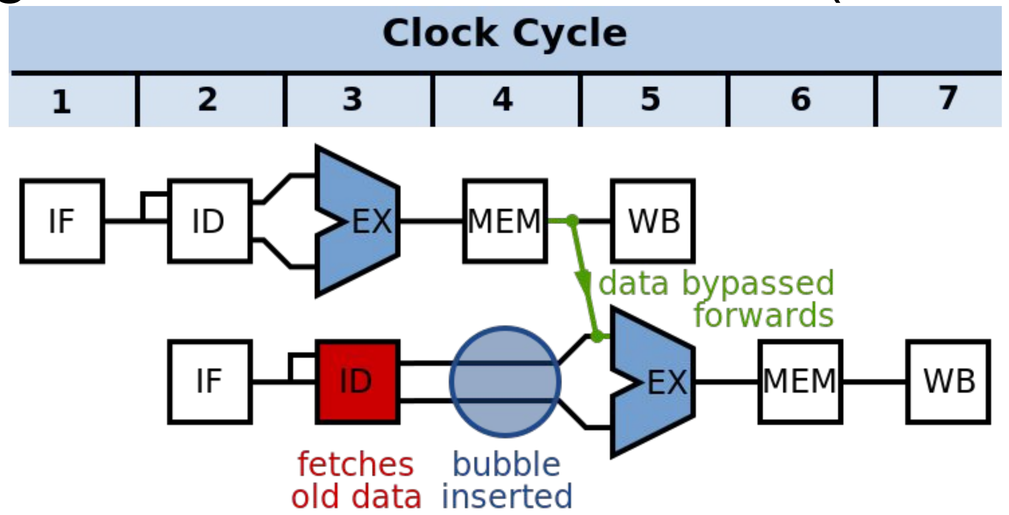
- If we look closely at the **pipeline**, we already have the result of i1 available in the 3rd clock cycle.
- It is not written back to the **register file**, but still the result is available at the output of the Execute stage.
- If we can detect the data dependency and then “forward” this data to the Execute stage input, then the next instruction can use the forwarded data instead of the data from the Decode stage. Thus data hazard is mitigated! The idea looks like this:



Classic RISC Pipeline

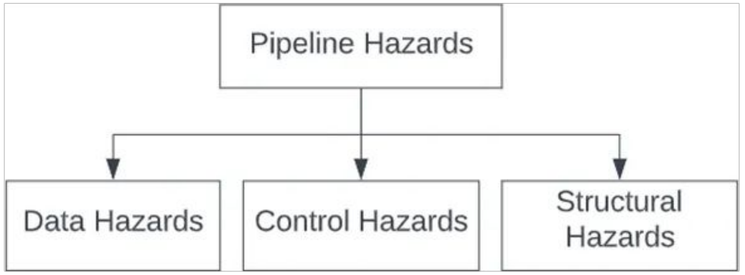


- The example shows a bubble being inserted into a classic RISC **pipeline**.
- At the 4th stage, it cannot do forwarding.
- Without a bubble, the EX stage (3rd stage) only has access to the output of the previous EX stage. Thus, adding a bubble resolves the time dependence without needing to propagate data backwards in time (which is impossible).



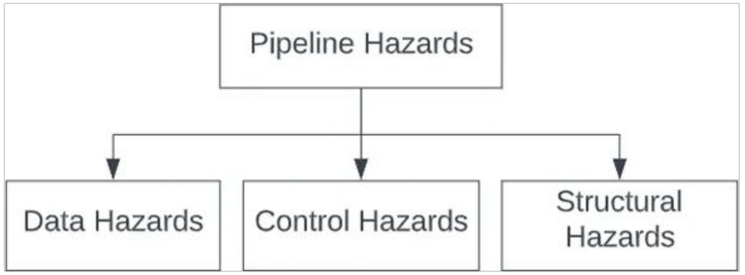
source: https://en.wikipedia.org/wiki/Pipeline_stall

Control Hazards



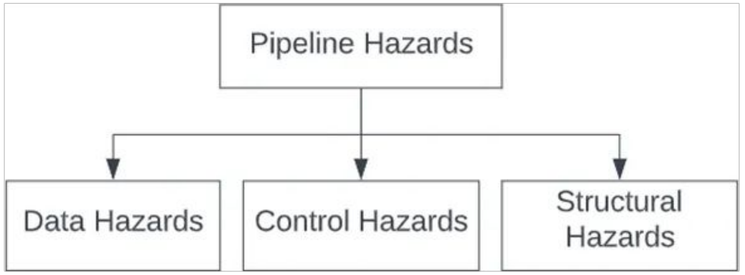
- Control Hazards are caused by Jump/Branch instructions.
- Jump/Branch instructions are the flow control instructions in the ISA of the CPU.
- When the control reaches a Jump/Branch instruction, the CPU has to make a decision whether to take the branch or not.
- One of the following actions should be taken.
  - Fetch the next instruction at PC+4 (branch not taken), or
  - Fetch the instruction at the branch target address (branch taken).

Control Hazards



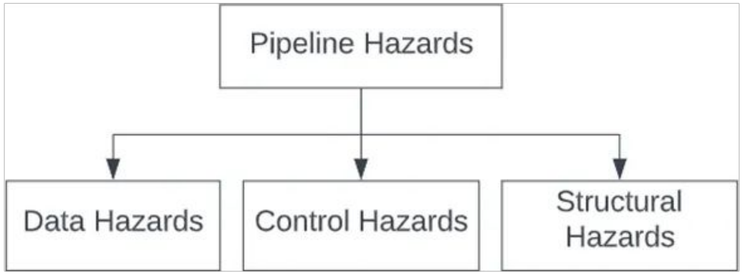
- Whether the decision was right or wrong is figured out only at the Execute stage, where the result of the branch instruction is computed.
- Depending on whether the branch is taken or not, the branch address (the address to which the CPU should branch out) is resolved.
- If the decision made earlier was wrong, all the instructions in the **pipeline** fetched and decoded until that clock cycle should be discarded.
- Because those instructions should not be executed at all.

Control Hazards



- This is done by flushing the **pipeline** and fetching the instruction at the branch address in the next clock cycle.
- Flushing invalidates the instructions and converts them into NOPs or bubbles.
  - This incurs a significant amount of clock cycles as a penalty.
- This is called a branch penalty.
- Control hazards thus have the worst effect on the performance of the CPU.

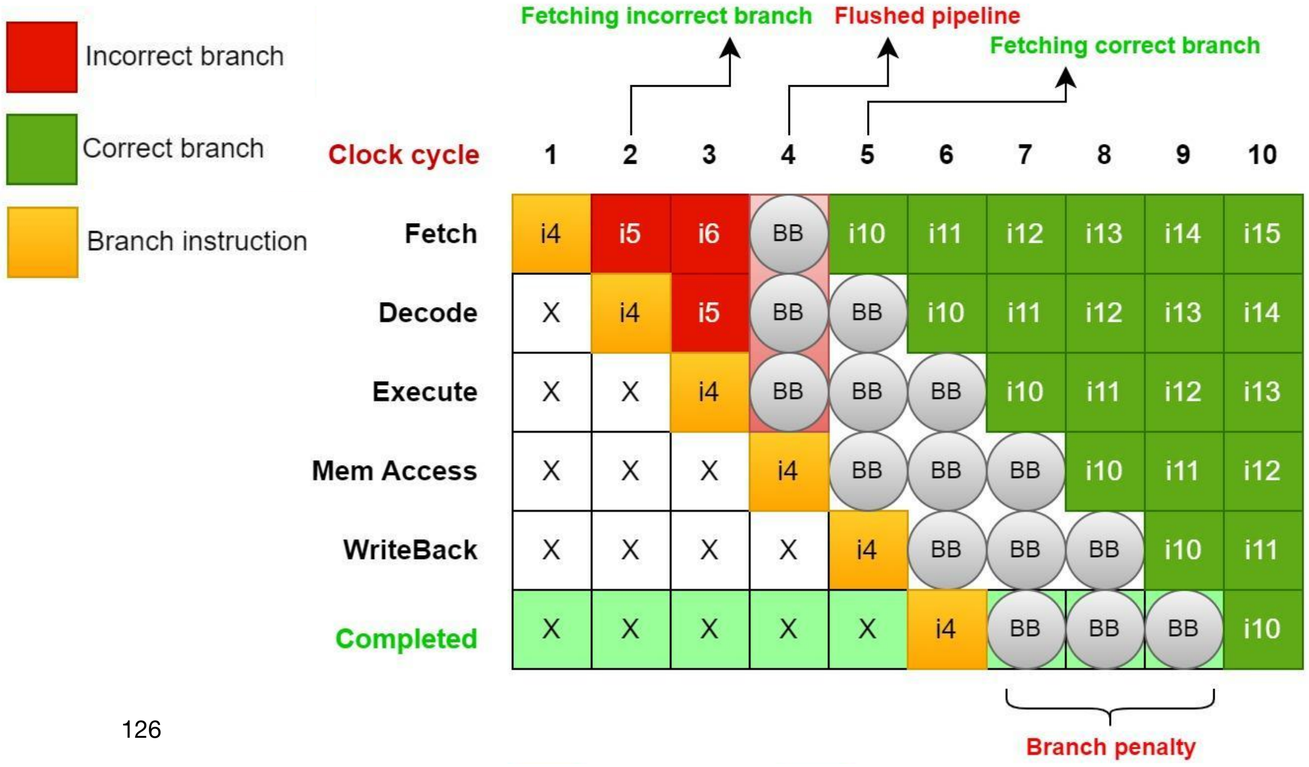
Flushing



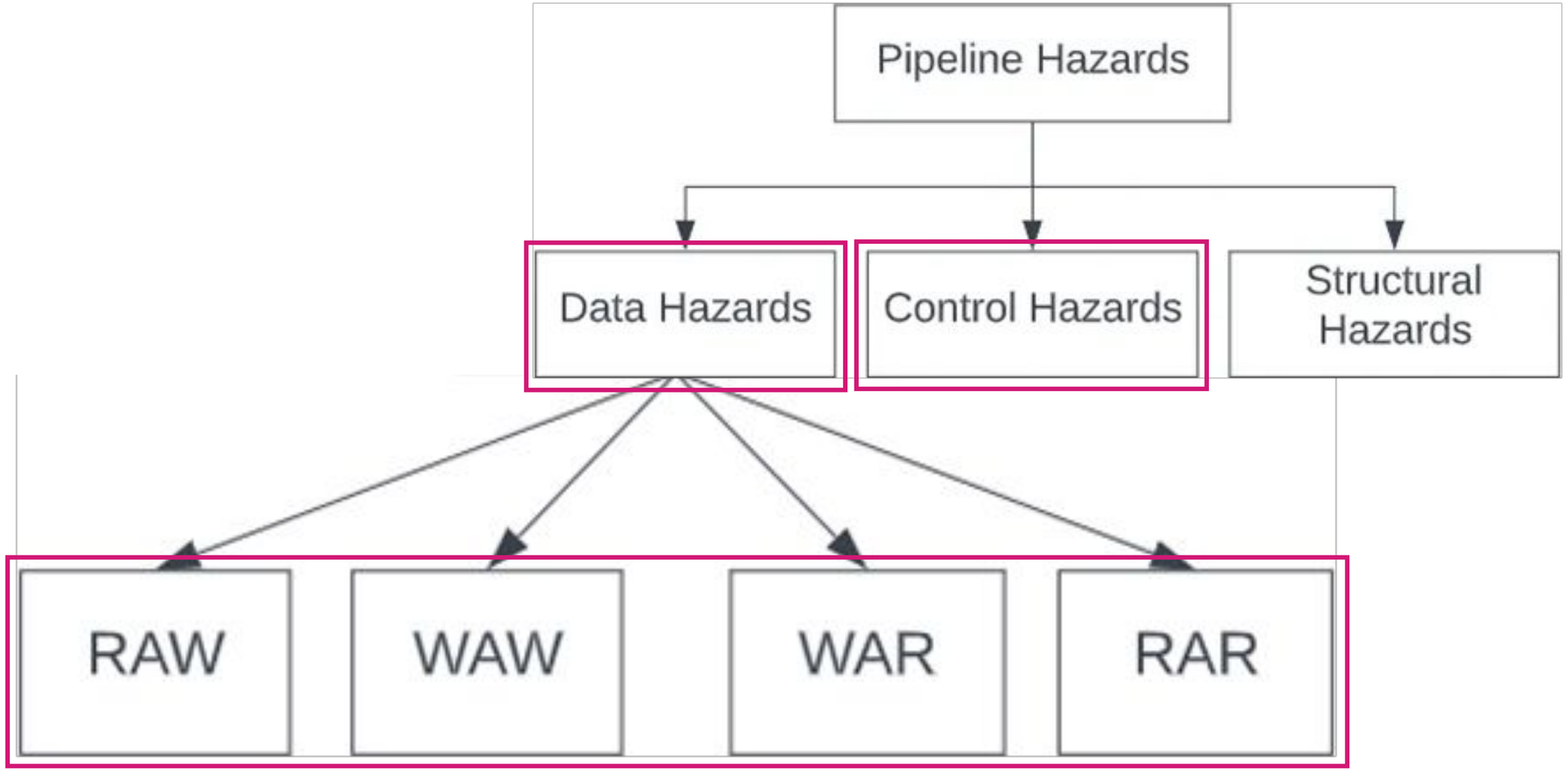
- Example, i10 completes the execution in 10th clock cycle, but it should have completed the execution in 7th clock cycle.
- A penalty of three clock cycles was incurred because the wrong branch (i5) was taken.

- Flush had to be done in the **pipeline** when this was identified by the Execute stage in the 4th clock cycle.
- How will this impact the CPU performance?
- If CPU has 30% branch instructions, CPI will become:
  - 50% performance down.

$$\text{CPI} = (\text{CPI}_{ideal} \times 0.70) + (3 \times 0.30) \approx 2$$



Pipeline Hazards



source: https://medium.com/@shrutika.gade22/pipeline-hazards-f6c317824b9f



Key Points: Pipeline Hazards

- **Data hazards: RAW, WAR, WAW dependencies between instructions**
  - RAW (true dependency) is the most common hazard type
- **Stall insertion adds bubbles to resolve dependencies**
  - Reduces throughput but ensures correct execution
- **Data forwarding bypasses pipeline stages to reduce stalls**
- **Control hazards from branch/jump require flushing or prediction**

References

References

Reference Materials

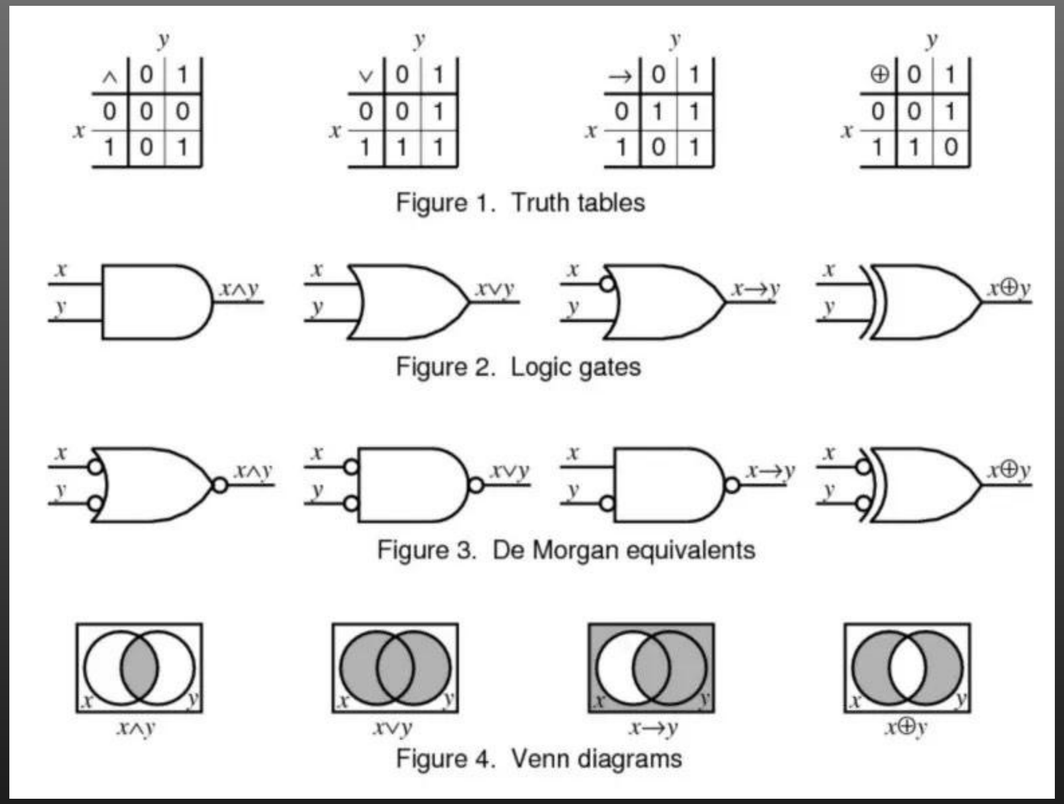
- William Stallings, “Computer Organization and Architecture Designing for Performance,” Tenth Edition, *Pearson*, 2016
- Tools:
  - https://cpulator.01xz.net/?sys=arm
  - https://tio.run/#
  - https://www.tutorialspoint.com/compile_assembly_online.php





Logic Gates

Unit 2: ALU & Register



$\wedge$	0	1
0	0	0
1	0	1

$\vee$	0	1
0	0	1
1	1	1

$\rightarrow$	0	1
0	1	1
1	0	1

$\oplus$	0	1
0	0	1
1	1	0

Figure 1. Truth tables

Figure 2. Logic gates

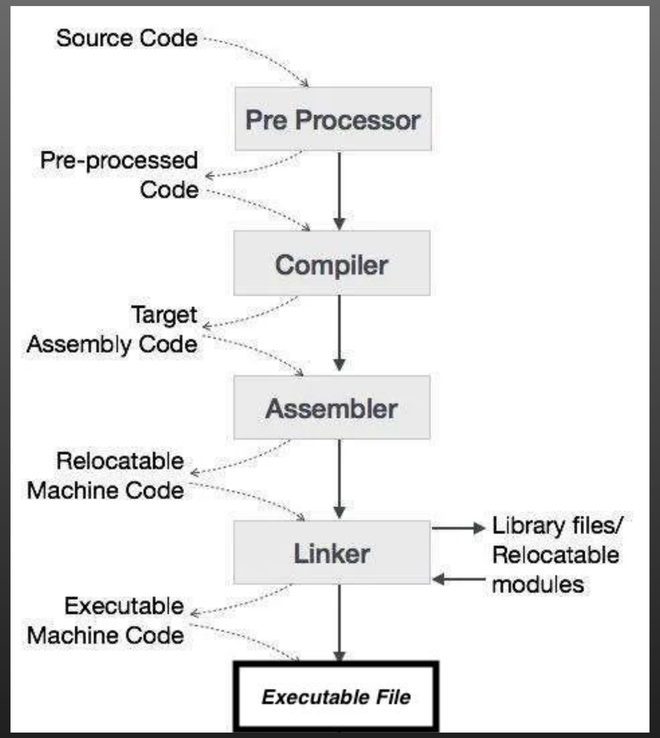
Figure 3. De Morgan equivalents

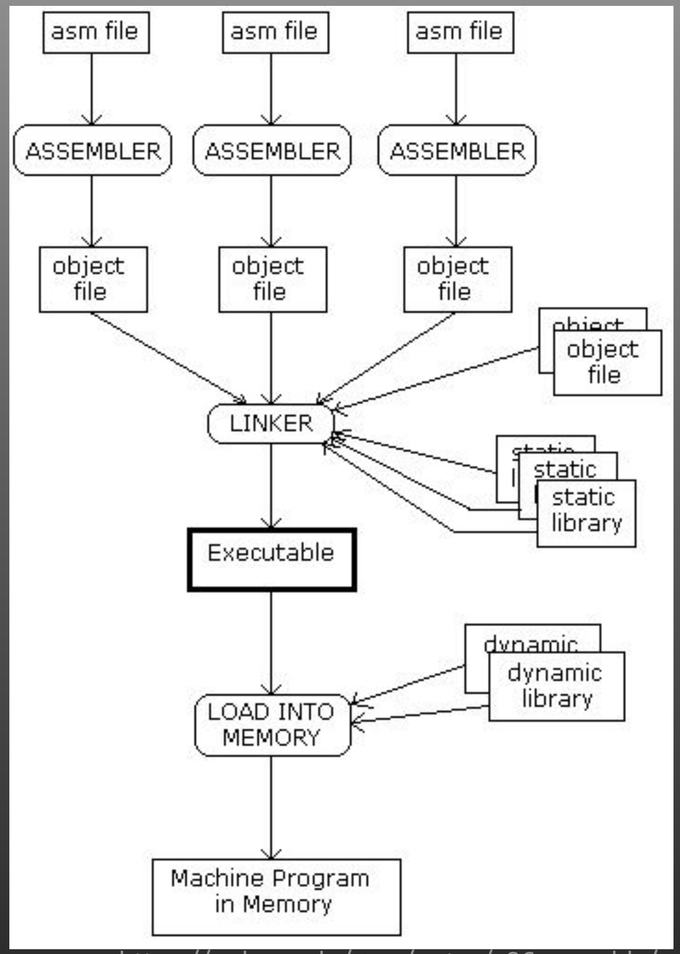
Figure 4. Venn diagrams

source: https://pablocorbalann.medium.com/programming-a-hello-world-in-assembly-from-the-first-line-to-the-end-x86-9c48fb499238

Assembly Execution

Unit 2: ALU & Register





source: https://cs.lmu.edu/~ray/notes/x86assembly/

source: https://pablocorbalann.medium.com/programming-a-hello-world-in-assembly-from-the-first-line-to-the-end-x86-9c48fb499238

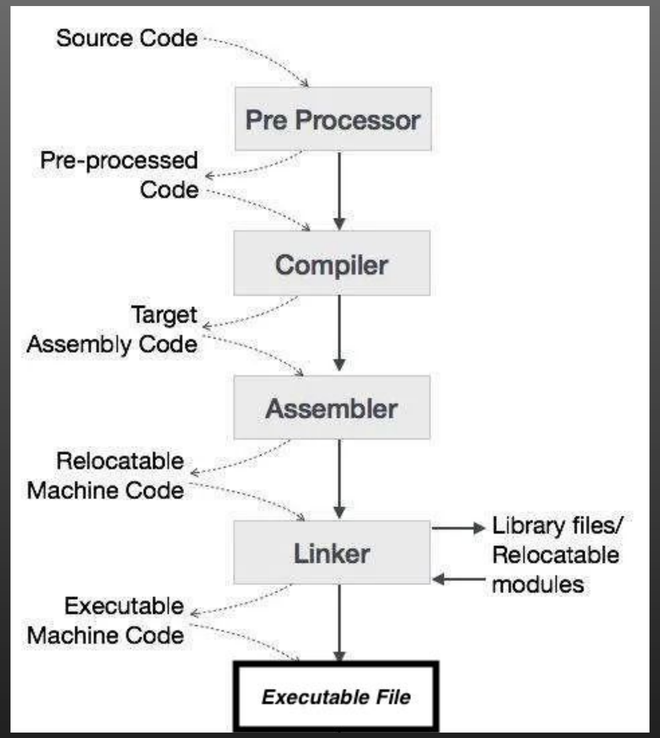
Processor Registers

Unit 2: ALU & Register

Processor Registers

8-bits	16-bits	32-bits	64-bits
al	ax	eax	rax
bl	bx	ebx	rbx
cl	cx	ecx	rcx
dl	dx	edx	rdx
sil	si	esi	rsi
dil	di	edi	rdi
bpl	bp	ebp	rbp
spl	sp	esp	rsp
r8b	r8	r8d	r8
r9b	r9	r9d	r9
r10b	r10w	r10d	r10
r11b	r11w	r11d	r11
r12b	r12w	r12d	r12
r13b	r13w	r13d	r13
r14b	r14w	r14d	r14
r15b	r15w	r15d	r15

source: https://medium.com/@notarioairlangga/simple-looping-in-assembly-summing-numbers-from-1-to-20-without-printing-an-introductory-aa6ecb7e98ed



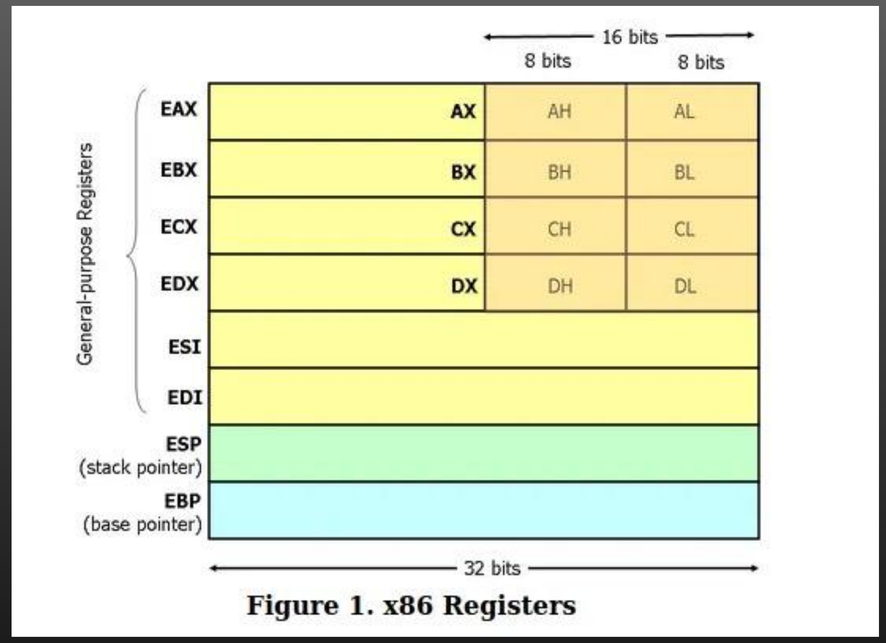


Figure 1. x86 Registers

source: https://pablocorbalann.medium.com/programming-a-hello-world-in-assembly-from-the-first-line-to-the-end-x86-9c48fb499238